

Appunti di Business Process Management (BPM)

Chiappini Sfeir

8 marzo 2026

Indice

1	Lezione 1: Introduzione al Business Process Management (BPM)	3
1.1	Contesto Operativo (Background)	3
1.2	Le Necessità Organizzative (What we need)	4
1.3	Definizione di Business Process	4
1.4	Interazioni tra Processi Aziendali	5
1.5	Business Process Management (BPM)	6
1.6	Stakeholder e Ruoli	7
1.7	Ciclo di Vita del Processo Aziendale (Business Process Lifecycle)	8
1.8	Identificazione dei Processi (Process Identification)	9
1.9	La Modellazione e le sue Proprietà	12
1.10	La Modellazione dei Processi (Process Modeling)	12
1.11	Architettura MOF (Meta Object Facility)	15
1.12	L'Analisi di Processo (Process Analysis)	16
1.13	Le Metriche di Prestazione (Performance Measures)	17
1.14	Tecniche per la Misurazione delle Prestazioni	18
1.15	Riprogettazione del Processo (Process Redesign)	19
1.16	Implementazione del Processo (Process Implementation)	20
1.17	Monitoraggio del Processo (Process Monitoring)	20
2	Lezione 2: Business Process Model and Notation (BPMN)	22
2.1	Le Componenti di un Linguaggio di Modellazione	22
2.2	Introduzione al BPMN	22
2.3	Il Metamodello BPMN	24
2.4	La Semantica del BPMN e il Concetto di Token	25
2.5	Gli Elementi di Base del BPMN	26
2.6	Il Nostro Primo Modello BPMN	27
2.7	I Gateway (Punti Decisionali)	28
2.8	Gateway Esclusivo (Exclusive Gateway - XOR)	28
2.9	Esempio Pratico: Spedizione di un Ordine	30
2.10	Gateway Parallelo (Parallel Gateway - AND)	31
2.11	Esempio Pratico: Controlli di Sicurezza all'Imbarco	33
2.12	Gateway Inclusivo (Inclusive Gateway - OR)	33

2.13	Esempio Pratico: Scomposizione dell'Ordine	35
2.14	Modellazione delle Risorse (Resource Modeling)	36
2.15	Flusso di Messaggio (Message Flow)	38
2.16	Artefatti (Artifacts)	39
2.17	Oggetti Dati (Data Objects)	40
2.18	Tipologie di Processi BPMN (BPMN Process Types)	41
2.19	Task Specializzati (Specialized Tasks)	43
2.20	Definizione degli Eventi (Event Definition)	44
2.21	Gateway Basato sugli Eventi (Event-Based Gateway)	50
3	Lezione 3: Metamodeling & Model Transformations	53
3.1	Il Ruolo della Modellazione e il Principio di Sostituibilità Limitata	53
3.2	Model-Driven Architecture (MDA) di OMG	55
3.3	Modello di Riferimento ODP (Open Distributed Processing)	57
3.4	Computation Independent Model (CIM)	58
3.5	Platform Independent Model (PIM)	59
3.6	Platform Specific Model (PSM)	60
3.7	Mappature (Mappings)	62
3.8	Principi e Standard della MDA di OMG	64
3.9	XMI in Eclipse	66
4	Capitolo 4: Model-to-Text Transformation	68
4.1	Introduzione alle Trasformazioni Model-to-Text (M2T)	68
4.2	Lo standard MOF M2T ed Acceleo	68
4.3	Approcci alla Generazione del Codice	69
4.4	Approfondimento sullo standard MOF M2T	70
4.5	L'Approccio basato su Template (M2T)	71
4.6	UML e le sue limitazioni	71
4.7	Object Constraint Language (OCL)	71
4.8	Vantaggi dell'approccio M2T	73
4.9	Struttura di un Generatore Acceleo	73
4.10	Collezioni OCL (OCL Collections)	75
4.11	Query e Servizi in Acceleo	76
4.12	Elementi Strutturali del Linguaggio (MTL)	78
5	Capitolo 5: Acceleo in Practice	80
5.1	Panoramica delle Attività (Outline)	80
5.2	L'Evoluzione dell'UML: I Profili (UML Profile)	80
5.3	Trasformazioni e Profili UML	81
5.4	Requisiti della Trasformazione M2T (Esempio Pratico)	83

1 Lezione 1: Introduzione al Business Process Management (BPM)

1.1 Contesto Operativo (Background)

Il nucleo fondamentale di ogni impresa risiede nella sua capacità di fornire prodotti e servizi ai propri clienti. Ogni prodotto o servizio erogato rappresenta l'output finale di un complesso di attività interconnesse. Queste attività possono essere eseguite secondo tre modalità principali:

- **Manualmente:** esclusivamente dai dipendenti, senza l'ausilio di tecnologie informatiche.
- **In modalità ibrida (Supportata da IT):** dai dipendenti con il supporto e l'integrazione di un Sistema Informativo (Information System).
- **Automaticamente:** eseguite in totale autonomia dai Sistemi Informativi, senza intervento umano diretto.

L'erogazione di tali prodotti e servizi non è casuale, ma è strettamente allineata agli **obiettivi di gestione aziendale** (Business Administration Goals), che tipicamente includono:

- L'incremento della soddisfazione del cliente.
- La riduzione dei costi operativi.
- Il miglioramento dell'efficienza generale.
- La contrazione dei tempi di esecuzione (Lead time).
- La minimizzazione del tasso di errore.

Affinché l'impresa possa raggiungere questi traguardi strategici, è imperativo che tutte le risorse aziendali, sia umane (dipendenti) che tecnologiche (sistemi informativi), operino in totale sinergia e cooperazione.

Chiarimento Accademico: Sinergia Risorse-Sistemi

In ambito BPM, la dicotomia tra lavoro umano e automazione non deve essere vista come un conflitto, ma come un'orchestrazione. I Sistemi Informativi moderni (come ERP o CRM) non si limitano a sostituire il lavoro manuale, ma fungono da "collante" informatico che instrada il lavoro verso il dipendente giusto al momento giusto, riducendo i colli di bottiglia e abilitando una reale misurazione delle performance aziendali.

1.2 Le Necessità Organizzative (What we need)

La presa di coscienza del contesto operativo fa emergere due necessità primarie per il raggiungimento degli obiettivi di business. È fondamentale:

1. **Organizzare strutturalmente** le molteplici attività aziendali.
2. **Comprendere a fondo** le logiche di interazione e dipendenza tra di esse.

La soluzione d'elezione a queste necessità risiede nell'adozione dell'approccio basato sui **Processi Aziendali** (Business Processes).

I processi aziendali agiscono come elemento facilitatore per la collaborazione tra le risorse d'impresa, garantendo che essa avvenga in modo sia *efficiente* (ottimizzazione delle risorse impiegate) sia *efficace* (raggiungimento dell'obiettivo prefissato).

Focus: L'Importanza della Modellazione dei Processi

I processi aziendali non possono essere gestiti efficacemente se rimangono una conoscenza "tacita", ovvero confinata nella mente di chi li esegue abitualmente. Per essere monitorati, analizzati e migliorati, i processi devono necessariamente essere resi "espliciti". Questo avviene attraverso i **modelli di processo** (Process Models): rappresentazioni grafiche e strutturate (es. tramite standard BPMN) che documentano in modo rigoroso il flusso di lavoro, i ruoli coinvolti, i dati scambiati e le regole di business.

1.3 Definizione di Business Process

Processo Aziendale (Business Process - BP)

- Un BP è un insieme di *attività logicamente correlate* che vengono eseguite in *modo coordinato*.
- Queste attività realizzano congiuntamente un *obiettivo di business*.
- Un BP è costituito da eventi, attività e punti di decisione.
- L'*esito (outcome)* ha un valore per almeno un cliente del BP.

Workflow o processo aziendale automatizzato

- Un processo che è automatizzato in tutto o in parte da un sistema software.
- Il sistema software trasferisce le informazioni da un partecipante all'altro in base alle dipendenze temporali e logiche definite nel modello di processo.

Chiarimento Accademico: BP vs Workflow

Mentre il "Business Process" rappresenta il concetto logico e organizzativo (che può anche essere completamente manuale), il termine "Workflow" si riferisce specificamente all'orchestrazione informatica del processo. L'automazione garantisce che il passaggio di consegne (handover) tra i vari attori avvenga sistematicamente

rispettando le regole e le sequenze (dipendenze temporali e logiche) modellate a monte.

1.4 Interazioni tra Processi Aziendali

Sebbene ogni Processo Aziendale (BP) sia tipicamente eseguito da una singola organizzazione, la realtà operativa richiede spesso una qualche forma di interazione con i processi di altre organizzazioni esterne. Per gestire queste dinamiche, si utilizzano due paradigmi fondamentali: l'orchestrazione e la coreografia.

Orchestrazione dei Processi Aziendali (Business process orchestration)

- Si riferisce ai BP eseguiti all'interno dell'organizzazione, i quali possono essere controllati e gestiti in modo **centralizzato**.
- *Metafora*: È simile a un'orchestra guidata dal suo direttore. Esiste un'entità centrale (il motore di orchestrazione) che coordina l'esecuzione, chiama i servizi e gestisce il flusso di lavoro interno.

Coreografia dei Processi Aziendali (Business process choreography)

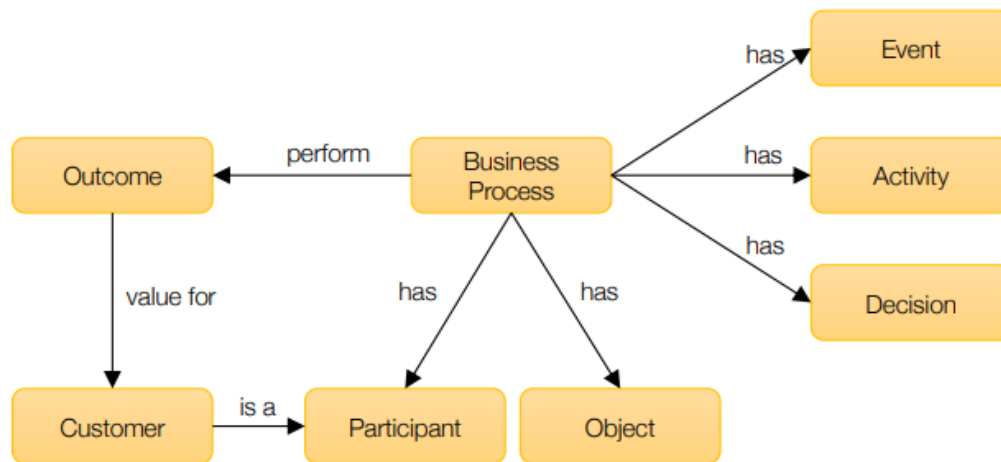
- Si applica ai BP che richiedono l'interazione con i processi appartenenti ad **altre organizzazioni**.
- Non essendoci un controllore centrale unico (ogni azienda mantiene la propria autonomia), la sincronizzazione tra i processi avviene tramite lo scambio di messaggi (*message passing*).
- *Metafora*: È simile a un gruppo di ballerini in cui ciascuno deve eseguire la propria danza in autonomia, ma nel rispetto di una coreografia comune concordata per garantire l'armonia globale.

Focus Accademico: Il paradosso del controllo

La differenza concettuale tra Orchestrazione e Coreografia risiede nel **dominio del controllo**.

- Nell'**Orchestrazione**, si ha una visione *end-to-end* "dall'alto", poiché il processo è interno all'azienda (es. approvazione di un rimborso spese aziendale).
- Nella **Coreografia**, ci si trova in un ambiente *peer-to-peer*. L'azienda conosce perfettamente i propri processi interni, ma vede i processi delle organizzazioni partner come una "scatola nera" (black box). L'interazione è regolata esclusivamente da protocolli condivisi e dai messaggi scambiati in input/output (es. l'interazione tra l'e-commerce di un'azienda e il sistema informativo del corriere espresso).

Business Process Key Concepts



1.5 Business Process Management (BPM)

Il **Business Process Management (BPM)** rappresenta la disciplina manageriale volta alla gestione e all'ottimizzazione dei processi.

- Una migliore comprensione delle operazioni aziendali e delle loro reciproche relazioni può essere attuata attraverso la rappresentazione esplicita dei processi aziendali.
- Il BPM si configura come un approccio "process-centric" (incentrato sul processo) mirato al *miglioramento delle prestazioni* (*improving performance*). Tale approccio combina attivamente le *tecnologie dell'informazione* (*information technologies*) con metodologie specifiche di processo e di governance.
- Il dominio del BPM include *concetti, metodi, tecniche e strumenti* (*concept, methods, techniques, tools*) necessari per supportare tutte le fasi del ciclo di vita di un Processo Aziendale (BP's lifecycle).

Chiarimento Accademico: L'Approccio Olistico del BPM

Il BPM non è un semplice strumento software, ma una disciplina organizzativa a tutto tondo. Il fatto che sia un approccio *process-centric* significa che l'azienda non viene più analizzata a "silos" (per dipartimenti isolati), ma come un flusso continuo trasversale che genera valore. L'integrazione tra Tecnologie dell'Informazione (IT) e metodologie di governance permette non solo di mappare teoricamente il processo, ma di controllarlo dinamicamente in ogni fase del suo ciclo di vita: dalla progettazione, all'esecuzione, fino al monitoraggio e all'ottimizzazione continua.

1.6 Stakeholder e Ruoli

L'implementazione del Business Process Management comporta la produzione di molteplici artefatti.

- Ogni artefatto possiede un *ambito (scope) differente* e uno specifico livello di astrazione, calibrati in base agli *stakeholder specifici* a cui sono destinati.

Gli stakeholder coinvolti nel ciclo di vita di un processo possono essere raggruppati in diversi ruoli chiave:

Ruoli Manageriali e di Business

- **Chief Process Officer (CPO):** Rappresenta un ruolo di top management (alta direzione). Ha la responsabilità di gestire globalmente tutti i processi aziendali (BP) dell'impresa.
- **Business Engineer:** È l'esperto del dominio di business. Il suo compito principale è definire gli obiettivi strategici che i BP devono raggiungere.
- **Process Designer:** È la figura tecnica che modella materialmente il BP, lavorando in costante interazione con gli altri stakeholder per raccogliere i requisiti.
- **Process Responsible (Responsabile di Processo):** È la figura incaricata di assicurare e supervisionare la corretta esecuzione del processo aziendale.

Ruoli Tecnici e Operativi

- **System Architect (Architetto di Sistema):** Si occupa di configurare e predisporre i sistemi informativi necessari per promulgare (enact) ed eseguire il BP.
- **Developers (Sviluppatori):** Sono i professionisti IT che si occupano dell'implementazione effettiva del BP all'interno dei sistemi software.
- **Process participants and workers (Partecipanti e Operatori):** Sono gli utenti finali che eseguono materialmente le attività a loro assegnate all'interno di una specifica istanza del BP.

Chiarimento Accademico: Artefatti e Istanze di Processo

Per comprendere appieno questa suddivisione dei ruoli, è essenziale chiarire due concetti chiave:

- **Artefatto:** Nel BPM, un artefatto è qualsiasi documento, modello o schema prodotto durante il ciclo di vita del processo. Il *Process Designer* creerà un artefatto visivo (es. un diagramma BPMN) ad alto livello di astrazione per il *Business Engineer*, ma produrrà un artefatto molto più tecnico e dettagliato (es. specifiche di integrazione dati) per i *Developers*.
- **Modello vs Istanza:** Il modello è lo schema generale del processo (il "progetto"). L'**istanza** è una singola esecuzione reale di quel modello. Ad esempio,

se il processo è "Gestione Ordini", il modello descrive *come* si gestiscono tutti gli ordini, mentre un'*istanza* è la gestione dello specifico ordine n. 4582 effettuato dal cliente Mario Rossi. Gli operatori (*workers*) lavorano sempre sulle *istanze*.

1.7 Ciclo di Vita del Processo Aziendale (Business Process Lifecycle)

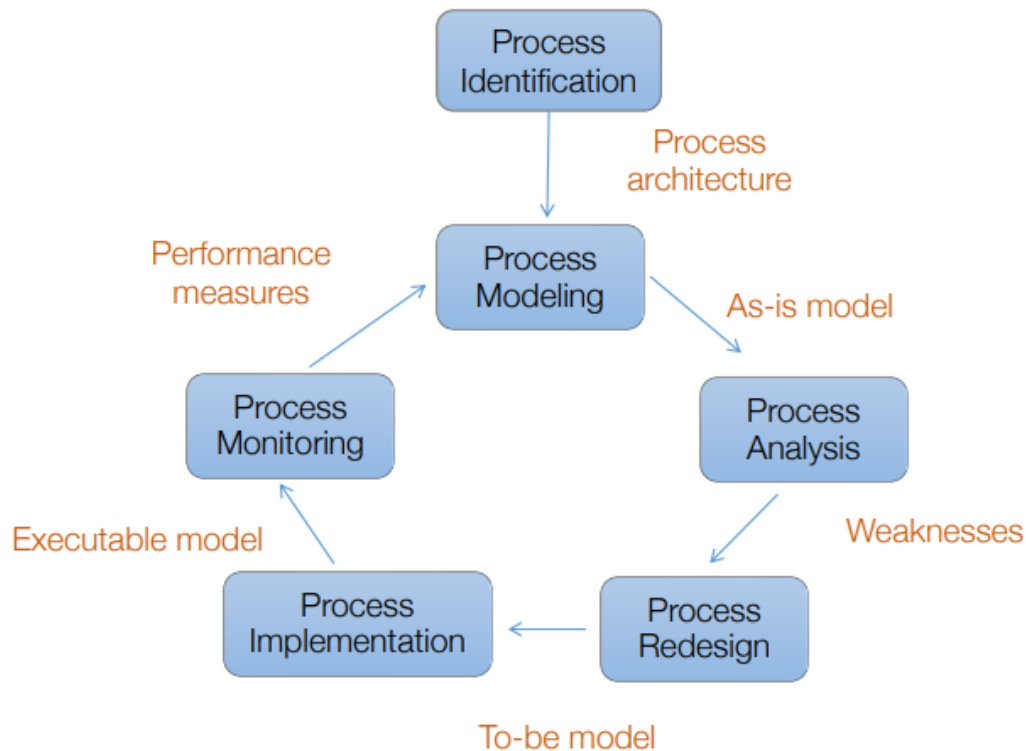
Il ciclo di vita di un processo descrive la metodologia strutturata attraverso cui i processi vengono concepiti, implementati e gestiti nel tempo.

- Il ciclo di vita di un BP è costituito da ***fasi*** (phases) che sono correlate tra loro secondo un modello di sviluppo a spirale.
- Ogni fase è composta da ***diverse attività*** (several activities) che presentano specifiche dipendenze logiche tra di loro.
- Per implementare il ciclo di vita di un BP, è inoltre possibile ricorrere ad approcci di tipo ***incrementale ed evolutivo***.

Chiarimento Accademico: Il Modello a Spirale e l'Evoluzione

In ambito BPM, l'adozione di un "modello di sviluppo a spirale" indica che la gestione dei processi non segue un percorso lineare e finito (come il classico modello a cascata o *waterfall*), ma rappresenta un ciclo continuo. Una volta che un processo è stato implementato e viene eseguito, i dati raccolti dal suo monitoraggio (le performance) diventano l'input per un nuovo ciclo di analisi e riprogettazione. L'approccio "incrementale ed evolutivo" è strettamente legato a questo concetto: permette alle organizzazioni di non dover stravolgere interamente un processo in un'unica volta, ma di applicare miglioramenti gradualmente e continui (*Continuous Improvement*), riducendo il rischio operativo e aumentando l'agilità aziendale.

Business Process Lifecycle



1.8 Identificazione dei Processi (Process Identification)

La prima fase operativa per il team di BPM consiste nell'*identificazione dei processi* ritenuti rilevanti per l'organizzazione e nella mappatura delle *relazioni* che intercorrono tra di essi. Questo insieme di processi e interrelazioni costituisce la cosiddetta **architettura dei processi** (*process architecture*).

Misurazione del Valore e delle Prestazioni

Per determinare l'effettiva rilevanza di un processo strategico, è strettamente necessario *misurare il valore* che esso è in grado di generare. Affinché questa misurazione sia oggettiva, le metriche di prestazione del processo (*process performance measures*) devono essere definite in modo inequivocabile. Esse si articolano in tre categorie principali:

- **Metriche relative ai costi** (*Cost-related measures*).
- **Metriche relative alle tempistiche** (*Time-related measures*).
- **Metriche relative alla qualità** (*Quality-related measures*), che includono, ad esempio, i tassi di errore (*error rates*).

Fasi dell'Identificazione

L'identificazione strutturata dei processi si articola in due fasi successive:

1. **Designazione** (*Designation*).
2. **Valutazione** (*Evaluation*).

È fondamentale sottolineare che nessuna di queste due fasi preliminari comporta o richiede lo sviluppo di **modelli di processo dettagliati**.

Chiarimento Accademico: Architettura vs Modellazione di Dettaglio

In questa fase embrionale del ciclo di vita BPM, l'obiettivo non è descrivere "come" un processo viene eseguito (azione che richiederà un modello di dettaglio in una fase successiva), ma comprendere "quali" processi esistono e "quanto" impattano sul business.

- La **Designazione** si occupa di stilare un inventario logico dei processi aziendali (es. "Gestione Ordini", "Assunzione Personale").
- La **Valutazione** utilizza i criteri di costo, tempo e qualità precedentemente definiti per stabilire un ordine di priorità. Se, ad esempio, un processo presenta alti tassi di errore e incide pesantemente sui costi operativi, verrà classificato come prioritario per futuri interventi di riprogettazione o automazione.

L'insieme di questi processi, unito alle loro dipendenze, forma l'**Architettura dei Processi**, una mappa concettuale di altissimo livello utile al management strategico.

Fase di Designazione (Designation Phase)

L'obiettivo primario della fase di designazione è quello di ottenere una comprensione chiara dei processi dell'organizzazione e delle loro reciproche interrelazioni.

Un aspetto critico di questa fase riguarda il livello di dettaglio adottato: il numero di processi identificati deve necessariamente rappresentare un giusto compromesso (*right trade-off*). Si presentano due scenari operativi opposti:

- **Identificazione di Pochi Processi (*Few processes*):**
 - Ogni processo presenta un ambito (*scope*) molto ampio, inglobando un vasto numero di operazioni interne.
 - *Implicazioni:* Una riprogettazione del processo (*process redesign*) avrebbe un impatto massiccio sull'efficacia complessiva dell'organizzazione, ma la sua esecuzione risulterebbe estremamente complessa e rischiosa.
- **Identificazione di Molti Processi (*Many processes*):**
 - Ogni processo ha un ambito (*scope*) molto limitato e circoscritto.
 - *Implicazioni:* L'impatto strategico di una potenziale riprogettazione è nettamente inferiore, ma l'intervento di ottimizzazione risulta molto più facile da gestire e implementare.

Chiarimento Accademico: Il Dilemma della Granularità

Il problema del "giusto compromesso" descritto nella slide è noto in letteratura BPM come la scelta della **granularità** del modello. Se un'azienda definisce come unico macro-processo la "Gestione dell'intero ciclo di vita del cliente" (scarsissima granularità, *pochi processi*), abbraccia troppe funzioni aziendali (marketing, vendite, produzione, logistica, customer care), rendendo quasi impossibile un'analisi di dettaglio e un'ottimizzazione mirata. Viceversa, se definisce come processo a sé stante l'azione "Stampare l'etichetta di spedizione" (eccessiva granularità, *molti processi*), il team di analisti si perderà in una micro-gestione frammentata, smarrendo la visione olistica della catena del valore. La designazione mira a trovare il livello di astrazione perfetto per la governance aziendale.

Fase di Valutazione (Evaluation Phase)

La fase di valutazione parte dal presupposto fondamentale che i processi devono essere disposti in ordine di priorità, in quanto non rivestono tutti la medesima rilevanza per l'organizzazione.

- I processi suscettibili di generare perdite o di esporre l'azienda a rischi devono essere attentamente analizzati ai fini di una loro potenziale **sistemazione/consolidamento**, **dismissione** (decommissioning) o **eliminazione** totale.

I criteri guida per condurre in modo strutturato questa fase di valutazione si basano su tre parametri principali:

- **Importanza (*Importance*)**: Consiste nel selezionare quei processi che esercitano il maggiore impatto nel raggiungimento degli obiettivi strategici dell'azienda.
- **Disfunzione (*Dysfunction*)**: Mira a individuare i processi che si trovano in uno stato di profonda criticità operativa. Questo perché i processi maggiormente "malati" sono anche quelli che trarranno il massimo profitto e beneficio da un'iniziativa di BPM.
- **Fattibilità (*Feasibility*)**: Stabilisce che il BPM dovrebbe concentrarsi su processi idonei, per i quali risulti ragionevole, logico ed economicamente sostenibile aspettarsi di ottenere dei benefici a valle dell'intervento.

Chiarimento Accademico: Il framework Importanza-Disfunzione-Fattibilità

Questo modello di valutazione a tre dimensioni è uno strumento cruciale per evitare la dispersione delle risorse aziendali (tempo, budget e personale) in progetti di miglioramento inefficaci.

- L'**Importanza** valuta l'allineamento strategico (es. il processo di customer care per un'azienda che punta sulla fidelizzazione).
- La **Disfunzione** funge da "diagnosi clinica": si interviene dove il dolore (inefficienza, colli di bottiglia, reclami) è più acuto.

- La **Fattibilità** funge da filtro di realtà (reality check): per quanto un processo possa essere strategico e disfunzionale, se l'azienda non possiede il budget, l'infrastruttura IT o la cultura organizzativa per modificarlo, l'iniziativa di riprogettazione è destinata a fallire. Un progetto BPM di successo si posiziona all'intersezione ottimale di queste tre variabili.

1.9 La Modellazione e le sue Proprietà

La modellazione rappresenta uno strumento metodologico fondamentale per *gestire la complessità*. I suoi scopi principali sono due:

- Comprendere a fondo il comportamento del mondo reale.
- Identificare e prevenire potenziali problemi o criticità.

Un *modello* è una rappresentazione astratta creata con l'intento di catturare aspetti specifici della realtà, nel rispetto di tre proprietà fondamentali:

- **Mappatura (*Mapping*):** Il modello deve riflettere fedelmente gli *aspetti chiave* del mondo reale.
- **Astrazione (*Abstraction*):** Il modello deve catturare unicamente gli aspetti rilevanti per lo scopo della modellazione, *astruendo da quelli irrilevanti* (ovvero omettendoli intenzionalmente per semplificare l'analisi).
- **Scopo (*Purpose*):** La struttura del modello dipende strettamente dal *pubblico di riferimento* (*target audience*). Generalmente si identificano due scopi principali per la modellazione dei processi.

Chiarimento Accademico: Il Paradosso dell'Astrazione

Un modello, per essere utile, deve necessariamente essere "sbagliato" o, meglio, incompleto. Se una mappa geografica (un modello spaziale) mappasse in scala 1:1 ogni singolo albero e sasso (mancanza di astrazione), sarebbe illeggibile e inutile. L'astrazione permette agli analisti di processo di concentrarsi solo sulle informazioni rilevanti (es. passaggi di stato, scambi di documenti), ignorando i dettagli superflui che complicherebbero la lettura del diagramma.

1.10 La Modellazione dei Processi (Process Modeling)

La modellazione dei processi (nota in letteratura anche come *process discovery* o *process design*) ha l'obiettivo primario di *comprendere* in dettaglio il funzionamento del processo aziendale.

- Il risultato operativo (*outcome*) di questa attività è la produzione di uno o più *modelli di processo as-is* (così com'è). Questi documenti fotografano lo stato attuale di ciascun processo rilevante precedentemente identificato.

- I modelli *as-is* riflettono la comprensione del lavoro e delle dinamiche operative così come percepite e vissute dalle persone all'interno dell'organizzazione.

Per ridurre drasticamente l'ambiguità tipica del testo libero (*free-form text*), i modelli assumono frequentemente la forma di **diagrammi** (diagrammi di flusso o *flowcharts*):

- Il diagramma deve avvalersi di una notazione grafica standard che possa essere compresa da tutti gli stakeholder coinvolti nel progetto.
- È comunque consentito e incoraggiato fornire una descrizione testuale complementare a supporto del modello visivo.

Focus: "As-Is" vs "To-Be"

La definizione del modello **As-Is** (stato dell'arte o situazione attuale) è il pilastro della fase di Process Discovery (scoperta del processo). Non si può migliorare un processo se prima non si comprende esattamente come viene eseguito oggi, comprese le sue inefficienze. Solo dopo un'attenta analisi dell'As-Is, il team BPM potrà progettare il modello **To-Be** (situazione futura o desiderata), che incorpora le ottimizzazioni, l'automazione e le soluzioni alle disfunzioni riscontrate.

Le Fasi Operative della Modellazione

L'esecuzione pratica della modellazione di processo si sviluppa attraverso quattro fasi metodologiche consecutive:

1. **Formazione del team (*Setting team*):** Assemblare all'interno dell'azienda un gruppo di lavoro con la specifica responsabilità di operare e studiare il processo.
2. **Raccolta delle informazioni (*Gathering information*):** Fase di indagine sul campo per costruire una solida comprensione delle dinamiche del processo.
3. **Modellazione (*Modeling*):** La creazione fisica o digitale vera e propria del modello di processo.
4. **Assicurazione della Qualità (*Assuring quality*):** La fase di verifica per garantire che il modello di processo sviluppato soddisfi appieno i criteri di qualità stabiliti (es. conformità sintattica della notazione, correttezza semantica).

Approfondimento Fase 1: Formazione del Team (*Setting Team*)

Nella prassi della modellazione di processo, si osserva tipicamente il coinvolgimento di due ruoli fondamentali, i quali possiedono competenze strettamente complementari:

- **Analisti di Processo (*Process Analysts*):**
 - Hanno piena familiarità con i linguaggi di modellazione e la creazione di diagrammi.
 - Generalmente, possiedono una comprensione solo superficiale o limitata dello specifico processo che deve essere modellato.

- Dipendono inevitabilmente dalle informazioni fornite dagli esperti di dominio.

- **Esperti di Dominio (*Domain Experts*):**

- Possiedono una conoscenza profonda e radicata su come il processo viene materialmente eseguito.
- Possono coincidere con varie figure aziendali: un partecipante al processo, il proprietario del processo (process owner) o un manager.
- Non hanno familiarità con i rigidi linguaggi di modellazione formale.

Competenze (Skills)	Process Analyst	Domain Expert
<i>Capacità di Modellazione</i>	Alta (Hi)	Bassa (Low)
<i>Conoscenza del Processo</i>	Bassa (Low)	Alta (Hi)

Chiarimento Accademico: Il Gap Semantico

La netta separazione tra Analisti ed Esperti di Dominio esiste per gestire il cosiddetto "gap semantico". L'Esperto ragiona in termini di pratica aziendale quotidiana (linguaggio di business e "knowledge" tacita), mentre l'Analista ragiona in logica di flussi, vincoli e sistemi (linguaggio tecnico e "knowledge" esplicita). La fase di modellazione è di fatto un delicato processo di traduzione interlinguistica che richiede una forte cooperazione.

Approfondimento Fase 2: Raccolta delle Informazioni (Gathering Information)

Per acquisire la comprensione necessaria a modellare il processo, la letteratura identifica tre principali famiglie di tecniche di "scoperta" (*discovery techniques*):

1. **Basate su Evidenze (*Evidence-based*):** Includono l'analisi della documentazione esistente, l'osservazione diretta sul campo (shadowing) o la scoperta automatica del processo (*process mining* tramite i log dei sistemi informativi).
2. **Basate su Interviste (*Interview-based*):** Colloqui strutturati o semi-strutturati condotti con logica in avanti (*forward*, dalla prima all'ultima attività) o a ritroso (*backward*, partendo dall'output).
3. **Basate su Workshop (*Workshop-based*):** Sessioni collaborative di gruppo che riuniscono contemporaneamente analisti e molteplici esperti di dominio.

Questi metodi differiscono sostanzialmente in base a quattro parametri chiave:

Parametro	Evidenze	Interviste	Workshop
<i>Oggettività (Objectivity)</i>	Alta	Media	Alta
<i>Ricchezza (Richness)</i>	Media	Alta	Alta
<i>Consumo di Tempo (Time)</i>	Basso	Medio	Medio
<i>Immediatezza del Feedback</i>	Bassa	Alta	Alta

Approfondimento Fase 3: Il Metodo di Modellazione (Modeling Method)

Una volta raccolte le informazioni, la stesura fisica del modello di processo procede tipicamente attraverso un metodo strutturato in cinque stadi sequenziali (*five stages*):

1. **Identificazione dei confini del processo (*Identify the process boundaries*):** Definire l'ambito operativo individuando con precisione quali eventi scatenano il processo (*trigger*) e quali sono i possibili esiti finali (*outcomes*).
2. **Identificazione di attività ed eventi (*Identify activities and events*):** Stilare un elenco delle principali azioni svolte e degli eventi intermedi, temporaneamente senza preoccuparsi del loro ordine cronologico.
3. **Identificazione delle risorse e dei passaggi di consegna (*Identify resources and handovers*):** Attribuire le responsabilità chiarendo chi (*worker* o sistema) è incaricato di eseguire le specifiche attività identificate.
4. **Identificazione del flusso di controllo (*Identify the control flow*):** Strutturare le logiche temporali stabilendo "quando" e "perché" eventi e attività vengono eseguiti. In questa fase si delineano le dipendenze d'ordine, i punti decisionali (bivi) e le eventuali ripetizioni (*loop*).
5. **Identificazione degli elementi aggiuntivi (*Identify additional elements*):** Arricchire il modello integrando la documentazione di supporto (artefatti), i dati in ingresso/uscita, l'interazione con i database e le procedure di gestione delle eccezioni o degli errori (*exception handlers*).

Focus: Approccio Top-Down

Il metodo in cinque fasi appena descritto adotta un classico approccio *Top-Down* (dall'alto verso il basso). Non si inizia mai disegnando le singole frecce del diagramma. Prima si circoscrive il "recinto" (i confini), poi si buttano all'interno i "mattoni" grezzi (attività ed eventi slegati), successivamente si assegnano i "murali" (le risorse) e solo alla fine si applica il "cemento" logico (il flusso di controllo) per costruire l'architettura finale, rifinandola poi con i dettagli (dati e documenti).

1.11 Architettura MOF (Meta Object Facility)

Per garantire che i modelli di processo (come pure i modelli software) siano rigorosi, standardizzati e interpretabili dai sistemi informatici, il settore IT si appoggia all'architettura MOF (Meta Object Facility).

Il MOF è un framework di astrazione strutturato in una rigida **gerarchia a 4 livelli** (4 layers). Ogni livello definisce le "regole" per la costruzione del livello immediatamente sottostante:

- **Livello M3 (Meta-metamodello):** È il livello di astrazione più alto e fondamentale. Definisce i concetti astratti universali, stabilendo, ad esempio, che "*Un metamodello è composto da entità e connessioni*". È il linguaggio condiviso da cui derivano tutti i linguaggi di modellazione successivi.

- **Livello M2 (Metamodello):** Definisce la grammatica e il vocabolario di uno specifico linguaggio di modellazione.
 - *In ambito BPMN (Processi):* Definisce che "Una sequenza è una relazione temporale tra due attività".
 - *In ambito Classi/UML (Software):* Definisce che "La generalizzazione è una relazione strutturale tra due classi".
- **Livello M1 (Modello):** È il livello in cui opera tipicamente l'analista (Process Designer), applicando le regole del livello M2 per descrivere uno scenario specifico.
 - *In ambito Processi:* Si disegna il modello affermando che "Il processo analizzato è costituito dalla sequenza dell'attività A seguita dall'attività B".
 - *In ambito Software:* Si modella stabilendo che "La Classe A è una superclasse della Classe B".
- **Livello M0 (Istanza):** Rappresenta il livello della realtà materiale ed esecutiva.
 - *In ambito Processi:* È il "Processo aziendale reale" eseguito in un dato momento da un operatore (es. l'ordine effettivo del cliente Rossi).
 - *In ambito Software:* È la "Implementazione software" vera e propria in esecuzione.

Chiarimento Accademico: Perché 4 Livelli di Astrazione?

La struttura MOF può sembrare un'inutile complicazione teorica, ma è essenziale per l'interoperabilità dei sistemi. Il **BPMN** (*Business Process Model and Notation*) non è un semplice "disegno", ma un Metamodello (Livello M2). Se definiamo le regole del BPMN al livello M2, possiamo usare dei software grafici per tracciare un diagramma (Livello M1). Poiché quel diagramma rispetta le regole rigorose di M2, il Sistema Informativo aziendale (motore di workflow) è in grado di leggerlo matematicamente e di trasformarlo automaticamente in codice eseguibile e in un processo reale governato dall'IT (Livello M0). L'astrazione garantisce che il disegno tecnico corrisponda a un'istruzione informatica inequivocabile.

1.12 L'Analisi di Processo (Process Analysis)

Una volta che il processo è stato correttamente identificato e modellato, si passa alla fase di analisi. Questa fase si concentra sull'*identificazione e sull'analisi delle criticità (issues)* che affliggono i processi operativi.

- Comprendere le *cause principali* (main causes) che generano risultati negativi permette di identificare la strada migliore per risolvere tali problematiche.
- L'output finale di questa fase analitica è una *raccolta strutturata* di tutte le criticità riscontrate.
- Affinché l'analisi sia oggettiva e non basata su semplici sensazioni, ogni criticità deve essere rigorosamente quantificata utilizzando *metriche di prestazione* (performance measures).

1.13 Le Metriche di Prestazione (Performance Measures)

In linea generale, le prestazioni di un processo aziendale vengono misurate tenendo in considerazione **tre aspetti fondamentali**:

1. **Tempo** (*Time*)
2. **Costo** (*Cost*)
3. **Qualità** (*Quality*)

Ciascuno di questi tre macro-aspetti può essere ulteriormente raffinato e declinato in specifiche metriche, denominate **KPI** (*Key Performance Indicator*). Un KPI è una quantità numerica che può essere calcolata *in modo inequivocabile* (unambiguously) partendo dai dati e dalle informazioni generate dal processo.

Chiarimento Accademico: Il Triangolo del Diavolo (Devil's Triangle)

Tempo, Costo e Qualità non sono variabili indipendenti, ma compongono quello che in letteratura gestionale è noto come "Triangolo del Diavolo". Spesso, l'ottimizzazione di un parametro peggiora inevitabilmente gli altri. Ad esempio, ridurre i tempi di esecuzione e aumentare la qualità di un servizio richiede quasi sempre un aumento esponenziale dei costi (es. assumere personale più qualificato o acquistare software migliori). L'Analisi di Processo serve ai manager per trovare l'equilibrio (trade-off) più sostenibile per l'azienda.

1. Misurazione del Tempo (Time Measure)

Il tempo è storicamente la metrica di performance più utilizzata. Il tempo di prestazione di un processo si divide in tre componenti principali:

- **Tempo di Ciclo o di Attraversamento** (*Cycle time or throughput time*): Rappresenta il tempo totale necessario per gestire una singola "pratica" (case) dall'inizio alla fine del processo. È la metrica di riferimento quando si affronta una riprogettazione del processo.
- **Tempo di Elaborazione o di Servizio** (*Processing time or service time*): Indica il tempo effettivamente speso dalle risorse lavorative (es. i partecipanti umani o le applicazioni software invocate) per la gestione attiva della pratica.
- **Tempo di Attesa** (*Waiting time*): Rappresenta il tempo in cui il processo rimane inattivo (*idle mode*). Include il tempo di coda (in attesa che una risorsa si liberi), il tempo di sincronizzazione e l'attesa per ricevere un input da un altro partecipante.

2. Misurazione del Costo (Cost Measure)

Per misurare in modo accurato l'impatto economico, il costo totale del processo viene scomposto in due categorie:

- **Costi Fissi (*Fixed costs*):** Sono quei costi che non subiscono variazioni in base all'intensità o al volume di elaborazione del processo. Esempi tipici includono l'utilizzo dell'infrastruttura di base o la manutenzione annuale dei sistemi informativi.
- **Costi Variabili (*Variable costs*):** Sono quei costi direttamente correlati a una quantità variabile (ai volumi). Esempi includono il livello delle vendite, la quantità di merci acquistate o il numero di nuove assunzioni effettuate.

3. Misurazione della Qualità (Quality Measure)

Il concetto di qualità può essere analizzato da due prospettive distinte: quella del cliente e quella del partecipante interno. Di conseguenza, si suddivide in:

- **Qualità Esterna (*External quality*):** Riguarda la soddisfazione del cliente rispetto al prodotto o all'erogazione del processo. Include la *soddisfazione sul prodotto* (in termini di rispetto delle specifiche o delle aspettative) e la *soddisfazione sul processo* (come il cliente ha percepito l'esecuzione del servizio).
- **Qualità Interna (*Internal quality*):** È la qualità valutata dal punto di vista del partecipante (l'operatore aziendale). Si traduce nel livello di controllo che egli ha sull'attività e sulla quantità di variazioni (imprevisti o deviazioni) riscontrabili nel flusso del processo.

1.14 Tecniche per la Misurazione delle Prestazioni

Per calcolare o stimare i KPI appena descritti, gli analisti si avvalgono di tre principali metodologie matematiche e informatiche:

1. **Analisi del flusso (*Flow analysis*):** È una tecnica che permette di stimare le prestazioni complessive (globali) di un processo a partire dalla conoscenza delle prestazioni delle sue singole attività costituenti.
2. **Teoria delle code (*Queuing theory*):** Consiste nell'applicazione di tecniche matematiche volte ad analizzare i sistemi che presentano problemi di "contesa delle risorse" (*resource contention*), ovvero situazioni in cui più entità competono per l'utilizzo di una risorsa limitata, generando colli di bottiglia.
3. **Simulazione (*Simulation*):** Tramite appositi software, la simulazione esegue una moltitudine di "istanze virtuali" del processo, registrando accuratamente i dati per ogni singolo passaggio dell'esecuzione. Produce statistiche avanzate relative ai tempi di ciclo, ai tempi medi di attesa e all'utilizzo medio delle risorse.

Chiarimento Accademico: Analisi Statica vs Dinamica

Le prime due tecniche (**Flow Analysis** e **Queuing Theory**) rappresentano generalmente un'analisi **statica/matematica**. Sono utili e veloci, ma faticano a gestire scenari molto complessi con molte eccezioni aziendali. La **Simulazione**, invece, rappresenta un'analisi **dinamica**. Immettendo i dati "As-Is" in un motore di simulazione, il sistema "fa girare" virtualmente l'azienda per mesi o anni in pochi

secondi. Modificando i parametri (es. "Cosa succede se assumo 2 nuovi impiegati al centralino?"), l'analista può prevedere in modo granulare l'abbattimento del "Waiting Time" prima ancora di spendere budget reale per le assunzioni (*What-If Analysis*).

1.15 Riprogettazione del Processo (Process Redesign)

Una volta comprese le criticità (*issues*) tramite la fase di analisi, il ciclo di vita del BPM procede con la riprogettazione del processo.

- Questa fase si concentra sull'*identificazione e sull'analisi dei rimedi* (*remedies*) atti a risolvere le criticità emerse.
- Spesso esistono molteplici opzioni (*multiple options*) per affrontare un singolo problema. I vari rimedi possibili vengono accuratamente confrontati e valutati in base alle metriche di performance (*performance measures*) scelte in precedenza.
- L'obiettivo e l'output principale di questa fase è la proposta di una *versione riprogettata del processo*, nota come modello **To-Be** (situazione futura o desiderata).
- Analisi e riprogettazione sono fasi *intrinsecamente correlate* (*intricately related*): per ogni singola opzione di modifica ipotizzata durante la riprogettazione, deve essere condotta una corrispondente e preventiva analisi d'impatto.

Le Sfide della Riprogettazione (Process Redesign Challenge)

Modificare un processo aziendale consolidato è un'operazione complessa e delicata. Le principali sfide includono:

- **Resistenza al cambiamento:** I lavoratori (*workers*) cambiano abitudini con grande difficoltà, in quanto assuefatti a operare secondo metodologie preesistenti.
- **Impatto Tecnologico ed Economico:** I cambiamenti organizzativi richiedono quasi sempre pesanti modifiche ai Sistemi Informativi, comportando notevoli costi associati.
- **Impatto Esterno:** Le modifiche possono ripercuotersi anche sulle altre organizzazioni (partner, fornitori, clienti) che interagiscono con il processo, estendendo la complessità oltre i confini aziendali.

Chiarimento Accademico: L'Inerzia Organizzativa

La resistenza al cambiamento è uno dei principali motivi di fallimento dei progetti BPM. In letteratura gestionale, questo fenomeno è noto come *inerzia organizzativa*. Anche se il modello "To-Be" è matematicamente perfetto e promette riduzioni di costo del 30%, se le risorse umane percepiscono il cambiamento come una minaccia al proprio status o alla propria routine (es. paura di non saper usare il nuovo software o timore di licenziamenti), saboteranno, consapevolmente o meno, la sua adozione.

1.16 Implementazione del Processo (Process Implementation)

La fase di implementazione è il momento in cui i cambiamenti necessari per transitare dal modello "As-Is" al modello "To-Be" vengono materialmente eseguiti. Questa fase si divide in due direttrici parallele e fondamentali:

1. **Gestione del Cambiamento Organizzativo (*Organizational Change Management*):** L'insieme delle attività richieste per modificare concretamente il modo di lavorare (*way of working*) dei partecipanti coinvolti nel processo.
2. **Automazione del Processo (*Process Automation*):** Lo sviluppo e il rilascio in produzione (*deployment*) dei sistemi IT necessari per l'esecuzione tecnica del processo "To-Be".

Approfondimento: Gestione del Cambiamento Organizzativo

Per mitigare l'inerzia organizzativa vista in precedenza, il Change Management deve essere strutturato attraverso passaggi precisi:

- **Comunicazione:** È fondamentale spiegare ai partecipanti non solo *quali* cambiamenti verranno introdotti, ma soprattutto *quali saranno i benefici* derivanti da tali modifiche.
- **Pianificazione (*Change management plan*):** Definire con esattezza *quando* i cambiamenti avranno effetto e stabilire le modalità per *gestire la transizione* in modo morbido.
- **Formazione (*Training*):** Insegnare al personale il nuovo metodo di lavoro e monitorare l'assimilazione dei nuovi concetti operativi.

1.17 Monitoraggio del Processo (Process Monitoring)

Il BPM non si conclude con l'implementazione del nuovo processo. La gestione dei processi è uno **sforzo continuo** (*continuous effort*).

- Il monitoraggio prevede la raccolta e l'analisi dei dati rilevanti generati durante l'esecuzione quotidiana del processo, al fine di identificare la necessità di possibili aggiustamenti (*adjustments*).
- I parametri chiave monitorati includono: lo **stato attuale** del processo aziendale, le **eccezioni** (anomalie), i **tempi di esecuzione** e l'**utilizzo delle risorse**.
- Gli aggiustamenti si rendono necessari per garantire che il processo soddisfi costantemente le aspettative aziendali (*business expectations*), allineandosi agli obiettivi di performance prefissati.
- Attraverso il monitoraggio, è possibile rilevare tempestivamente la formazione di nuovi **colli di bottiglia** (*bottlenecks*), **errori ricorrenti** o qualsiasi **deviazione** rispetto al comportamento atteso (*expected behavior*).
- Una volta rilevata la deviazione, si procede con l'identificazione di nuovi rimedi, innescando di fatto un nuovo ciclo di vita del BPM.

Focus Tecnologico: Il BAM e il Process Mining

Nelle aziende moderne, il "Process Monitoring" è quasi totalmente automatizzato tramite cruscotti digitali (Dashboard) di **BAM** (***B**usiness **A**ctivity **M**onito-**r**ing*). Questi sistemi leggono in tempo reale i log dei software (ERP, CRM) e avvertono i manager se un KPI scende sotto la soglia di guardia. La deviazione dal "comportamento atteso" viene oggi analizzata tramite strumenti avanzati di *Process Mining*, che ricostruiscono visivamente le varianti del processo direttamente dai dati grezzi, chiudendo il ciclo a spirale verso una nuova fase di "Process Discovery".

2 Lezione 2: Business Process Model and Notation (BPMN)

2.1 Le Componenti di un Linguaggio di Modellazione

Prima di introdurre lo standard BPMN, è fondamentale comprendere come è strutturato un qualsiasi linguaggio di modellazione formale. Un linguaggio di modellazione si compone rigorosamente di tre parti interconnesse:

1. Sintassi (*Syntax*):

- Rappresenta l'insieme degli elementi di modellazione disponibili e le **regole** matematiche/logiche necessarie per combinarli correttamente.
- *Esempio*: La sintassi del BPMN include costrutti come le attività, gli eventi, i gateway (punti decisionali) e i flussi di sequenza.

2. Semantica (*Semantics*):

- Ha il compito di vincolare (*bind*) gli elementi sintattici a delle descrizioni testuali, conferendo loro un **significato preciso e inequivocabile**.
- Definisce il *comportamento* (*behavior*) effettivo degli elementi durante l'esecuzione del processo.

3. Notazione (*Notation*):

- È la veste grafica del linguaggio; definisce i **simboli visivi** specifici che rappresentano i vari elementi del modello.

Chiarimento Accademico: Sintassi vs Semantica

Per capire la differenza, pensiamo alla lingua italiana. "Il sasso mangia la mela" è una frase **sintatticamente** corretta (Soggetto + Verbo + Oggetto), ma **semanticamente** errata (un sasso non può compiere l'azione di mangiare). Nei modelli di processo accade lo stesso: potremmo unire due attività con una freccia in modo graficamente corretto (Sintassi), ma se la sequenza logica viola le regole di business aziendali o genera un paradosso temporale, il modello sarà semanticamente invalido e il software di automazione restituirà un errore.

2.2 Introduzione al BPMN

Il **Business Process Model and Notation (BPMN)** è lo standard internazionale di riferimento per la modellazione dei processi aziendali.

Evoluzione e Standardizzazione

Il BPMN è uno standard ufficiale mantenuto dall'**Object Management Group (OMG)**. La sua evoluzione ha attraversato diverse versioni chiave:

- Versione 1.0 (Rilasciata nel 2006)

- Versione 1.1 (2007)
- Versione 1.2 (2009)
- **Versione 2.0 (Rilasciata nel 2011)**

Gli Obiettivi del BPMN

Lo scopo primario del BPMN è fornire una notazione che sia universalmente **comprensibile** da un pubblico eterogeneo di stakeholder. È stato progettato per mettere in comunicazione:

- **Gli Analisti di Processo** (*BP analysts*).
- **Gli Sviluppatori IT** (*IT developers*).
- **Gli Operatori e i Manager aziendali** (*BP workers and managers*).

Le Caratteristiche Chiave del BPMN 2.0

Il successo del BPMN deriva dalla sua duplice natura, che bilancia accessibilità visiva e rigore informatico:

- **Familiarità Visiva:** Le figure di business e i manager si trovano molto a loro agio nel visualizzare i processi aziendali in un formato grafico basato sui classici **diagrammi di flusso** (*flow-chart format*).
- **Semantica di Esecuzione:** Nel BPMN, la semantica di esecuzione è **completamente formalizzata** (*fully formalized*).
- **Definizione Formale (Metamodello):** Con l'avvento della versione 2.0, il BPMN è stato dotato di una definizione formale rigorosa (*formal definition*) sotto forma di metamodello. Questa struttura fornisce una definizione precisa di tutti i costrutti e di tutte le regole necessarie per la creazione dei modelli.

Focus: Il "Ponte" tra Business e IT

La vera rivoluzione del BPMN 2.0 (rispetto alle versioni 1.x) risiede nel Metamodello e nella Semantica di Esecuzione formalizzata. Prima del BPMN 2.0, il manager disegnava un diagramma visivo (Business), e il programmatore doveva interpretarlo per scrivere il codice del software gestionale (IT). Spesso, l'interpretazione generava errori. Oggi, poiché il BPMN 2.0 ha un metamodello formalizzato (struttura XML sottostante), un diagramma disegnato correttamente da un Analista può essere letto *direttamente* e matematicamente da un motore software (*Workflow Engine*) che esegue il processo in automatico. Il BPMN non è più solo un "disegno su carta", ma un vero e proprio linguaggio di programmazione visuale.

2.3 Il Metamodello BPMN

L'introduzione di un metamodello formale all'interno dello standard BPMN porta con sé una serie di vantaggi fondamentali (*benefits*) per la disciplina della modellazione:

- **Formalizzazione di modelli ed entità (*Formalization of models and entities*):** Garantisce che ogni elemento grafico (es. un'attività o un evento) abbia una definizione univoca e rigorosa.
- **Formalizzazione delle relazioni (*Formalization of relationship between elements*):** Definisce le regole logiche e strutturali con cui i vari costrutti possono essere connessi tra loro.
- **Interoperabilità (*Interoperability*):** Permette ai modelli di essere esportati, letti e interpretati in modo coerente da software e piattaforme differenti.

Il Ruolo del Modellatore e degli Strumenti (Tools)

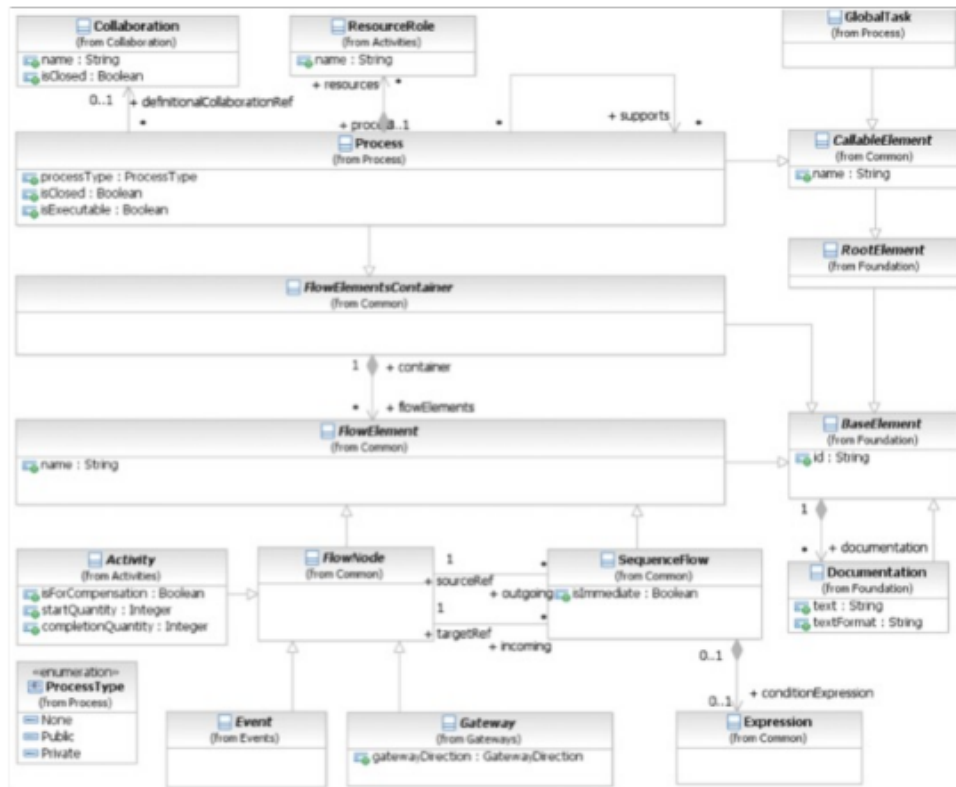
Nonostante la complessità tecnica che si cela dietro il metamodello formale, dal punto di vista operativo l'approccio è pensato per essere intuitivo:

- **Non è necessario** che l'analista o il modellatore (*modeler*) gestisca direttamente o manipoli il metamodello sottostante.
- È lo **strumento di modellazione** (*modeling tool*) a farsi carico di questo compito: il software si assicura automaticamente che il modello visivo disegnato dall'utente sia perfettamente conforme (*compliant*) alle regole dettate dal metamodello.

Chiarimento Accademico: Il Software come "Garante"

Quando un analista disegna un processo utilizzando un software di modellazione professionale (come Signavio, Bizagi, Camunda o Draw.io configurato per BPMN), sta interagendo esclusivamente con l'interfaccia visiva (il Livello M1 della gerarchia MOF). Se l'analista tenta di compiere un'azione logicamente non valida secondo lo standard (ad esempio, connettere un evento di fine direttamente a un evento di inizio senza attività intermedie), il software bloccherà l'operazione o segnalerà un errore di sintassi. Questo avviene perché il tool "conosce" il Metamodello BPMN (Livello M2) e agisce come un correttore sintattico in tempo reale, garantendo che il codice generato in background sia corretto e pronto per la futura automazione IT.

BPMN Metamodel (partial)



2.4 La Semantica del BPMN e il Concetto di Token

Per descrivere formalmente come si comportano gli elementi BPMN durante l'esecuzione di un processo, viene utilizzato il concetto teorico di *token* (gettone).

- Il comportamento "simulato" degli elementi del modello può essere definito descrivendo esattamente come essi interagiscono con il passaggio di questo token.
- È importante notare che il token *non fa parte delle specifiche standard* del linguaggio BPMN: agli strumenti di modellazione (tool) non è richiesto di implementare o visualizzare alcuna forma grafica del token. Ai fini didattici, si utilizza convenzionalmente il simbolo di un cerchio giallo contenente una 'T'.

Le Regole di Movimento del Token

- Il token attraversa i flussi di sequenza (*sequence flows*) e passa attraverso gli elementi che compongono il processo.
- L'attraversamento dei flussi di sequenza da parte del token avviene in modo *istantaneo*; pertanto, non vi è alcun lasso di tempo (time) associato al passaggio sui flussi.

- Quando il token arriva su un elemento (es. un'attività), può proseguire istantaneamente oppure essere trattenuto (subire un ritardo o *delay*), a seconda della natura e delle regole dello specifico elemento.

Chiarimento Accademico: L'Utilità del Token

Il concetto di "Token" deriva direttamente dalle Reti di Petri, un formalismo matematico per lo studio dei sistemi concorrenti. Immaginare un gettone che si muove lungo le frecce aiuta l'analista a verificare la tenuta logica del processo (*Token Analysis*). Se il token si sdoppia a un bivio (creando due processi paralleli) e poi i due rami non vengono riuniti correttamente, si genera un errore di semantica. I motori di automazione IT (*Workflow Engine*) ragionano esattamente così: muovono "gettoni virtuali" (istanze) da un operatore all'altro per tenere traccia dello stato di avanzamento di una pratica.

2.5 Gli Elementi di Base del BPMN

Flusso di Sequenza (Sequence Flow)

Il flusso di sequenza (rappresentato da una freccia a linea continua) connette gli elementi del modello mostrando il loro *ordine di esecuzione*.

- Ogni singolo flusso di sequenza possiede obbligatoriamente **una sola origine** (*source*) e **una sola destinazione** (*target*).
- Gli elementi del modello possono presentare uno o più flussi in entrata (*incoming*) e uno o più flussi in uscita (*outgoing*), sebbene solitamente ne abbiano solo uno per direzione.
- Origine e destinazione possono essere esclusivamente: attività, eventi o gateway.

Evento di Inizio (Start Event)

Rappresentato da un cerchio con bordo sottile singolo, indica l'inizio (*beginning*) del processo.

- Per sua natura logica, **non possiede flussi in entrata**.
- Un processo può teoricamente avere zero, uno o più eventi di inizio.
- Se l'evento di inizio non è presente nel diagramma, tutte le attività prive di flussi in entrata partiranno contemporaneamente.
- Se in un modello è presente un evento di fine, l'inclusione di almeno un evento di inizio diventa **obbligatoria**.

Evento di Fine (End Event)

Rappresentato da un cerchio con un bordo spesso (grassetto), indica il punto in cui il flusso del processo si conclude.

- Per sua natura logica, **non possiede flussi in uscita**.
- Un processo può teoricamente avere zero, uno o più eventi di fine.
- Se l'evento di fine non è presente, tutte le attività prive di flussi in uscita marcano la fine di uno specifico percorso (*process path*). In questo caso, il processo terminerà globalmente solo quando tutti i percorsi paralleli saranno stati completati.
- Se in un modello è presente un evento di inizio, l'inclusione di almeno un evento di fine diventa **obbligatoria**.

Task (Attività Elementare)

Rappresentato da un rettangolo con gli angoli arrotondati, il Task è **un'unità di lavoro** (*unit of work* o attività atomica), ovvero l'azione materiale che deve essere eseguita.

- Un Task viene utilizzato specificamente quando il lavoro all'interno del processo **non può essere ulteriormente scomposto** (*broken down*) a un livello di dettaglio inferiore.

Focus sulle Best Practice di Modellazione

Sebbene le rigide regole sintattiche del BPMN permettano formalmente di omettere gli Eventi di Inizio e Fine (i processi con "zero eventi" citati nelle slide), questa pratica è fortemente sconsigliata (*bad practice*). Un modello senza un chiaro inizio e una chiara fine genera ambiguità per gli stakeholder di business che devono leggerlo. La regola d'oro del Process Designer è tracciare sempre in modo esplicito gli eventi limite del processo, in modo da dichiarare inequivocabilmente il *trigger* (causa scatenante) e l'*outcome* (risultato finale).

2.6 Il Nostro Primo Modello BPMN

Per sintetizzare i concetti appena introdotti, è possibile costruire un semplice modello BPMN, il quale, in linea generale, possiede sempre quattro caratteristiche base:

1. Un punto di inizio (*A start point*).
2. Un punto di fine (*An end point*).
3. Una serie di attività (*Some activities*).
4. Le connessioni tra le attività (*Connections tra activities*).

Esempio di Flusso: Il diagramma esemplificativo mostra un flusso lineare che si innesca con l'Evento di Inizio "*Ordine di acquisto confermato*" (*Purchase order confirmed*).

Da qui, un Flusso di Sequenza conduce al primo Task: "Acquisizione indirizzo di spedizione" (*Get shipment address*). Segue in modo strettamente sequenziale il Task "Spedizione prodotto" (*Ship product*) e successivamente il Task "Ricezione pagamento" (*Receive payment*). Terminata quest'ultima unità di lavoro, un flusso in uscita guida il token verso l'Evento di Fine, sancendo che l' "Ordine è evaso" (*Order fulfilled*).



2.7 I Gateway (Punti Decisionali)

All'interno di un diagramma BPMN, i **Gateway** sono elementi utilizzati per controllare in che modo i flussi di sequenza interagiscono nel momento in cui convergono o divergono lungo il percorso del processo.

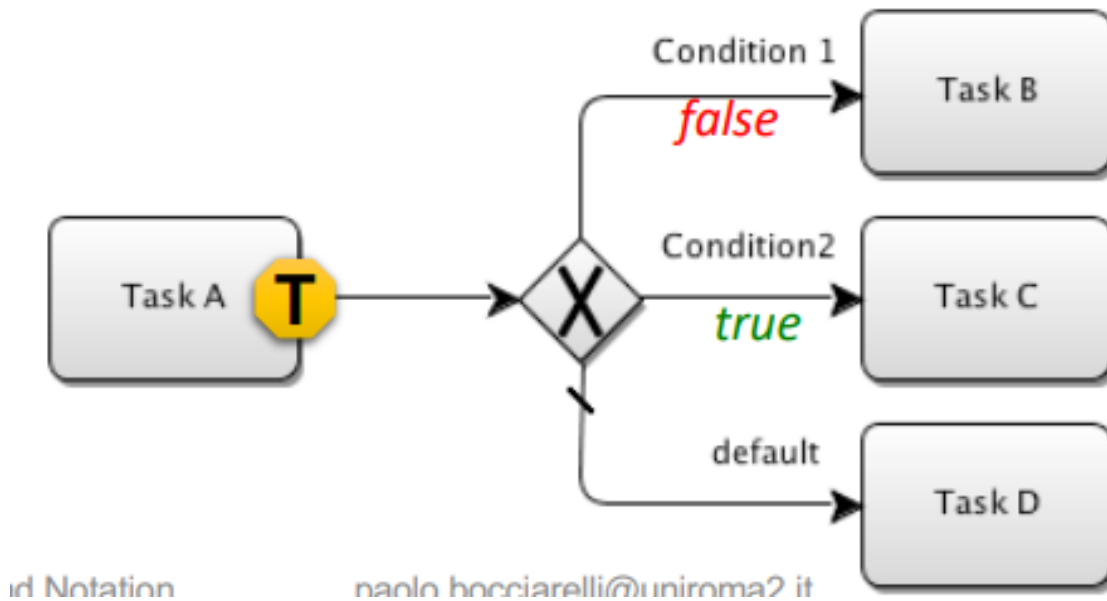
Da un punto di vista strutturale, si dividono in due macrotipologie:

- **Gateway di diramazione (*Split gateway*):** È il punto in cui il flusso del processo si divide in percorsi multipli. Presenta tipicamente **un solo** flusso di sequenza in entrata (*incoming*) e **molteplici** flussi di sequenza in uscita (*outgoing*).
- **Gateway di convergenza (*Merging gateway*):** È il punto in cui molteplici rami del processo si riuniscono in un unico flusso. Presenta **molteplici** flussi in entrata e **un solo** flusso di sequenza in uscita.

2.8 Gateway Esclusivo (Exclusive Gateway - XOR)

Il Gateway Esclusivo, rappresentato graficamente da un rombo che può essere vuoto oppure contenere una "X" al suo interno, implementa una logica decisionale strettamente mutuamente esclusiva.

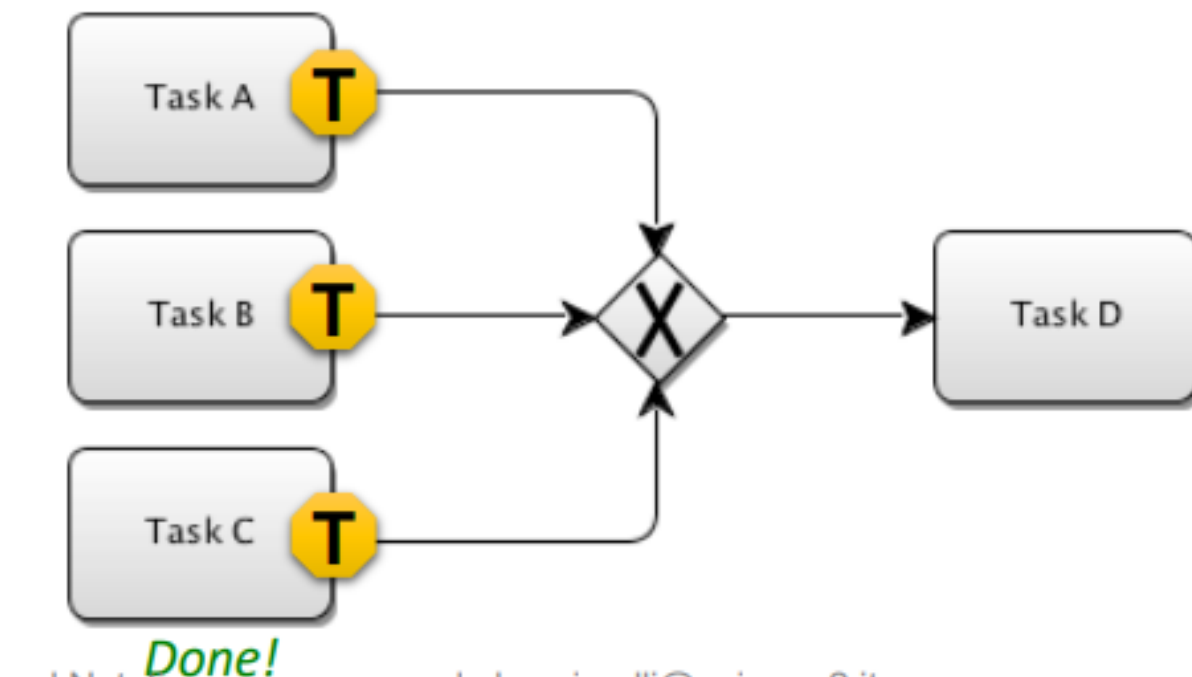
Divergenza Esclusiva (Diverging Exclusive Gateway)



Questo elemento crea dei percorsi alternativi (*alternative paths*) all'interno del flusso del processo.

- La decisione presa in questo punto viene formulata concettualmente come una domanda dotata di un set definito di risposte alternative.
- **Regola Logica (XOR):** Tra tutti i percorsi in uscita disponibili, *solo ed esclusivamente uno* può essere intrapreso.
- È buona norma definire un **percorso predefinito** (*default path*): questo ramo viene intrapreso automaticamente se nessuna delle espressioni condizionali poste sugli altri flussi viene valutata come "vera" (*true*).

Convergenza Esclusiva (Converging Exclusive Gateway)



Questo elemento viene utilizzato per unire (*merge*) i percorsi alternativi che sono stati creati in precedenza nel processo.

- Quando il token in arrivo da uno dei rami raggiunge questo gateway, viene instradato immediatamente verso il flusso di sequenza in uscita **senza alcuna sincronizzazione** (*without synchronization*).

Chiarimento Accademico: L'Assenza di Sincronizzazione

Affermare che la convergenza esclusiva avviene "senza sincronizzazione" significa descrivere il preciso comportamento del token. A differenza di altri gateway (es. il Gateway Parallelo - AND), il Gateway Esclusivo Convergente **non aspetta**. Poiché a monte (nella fase di Split) il token ha intrapreso *un solo* ramo, arriverà al gateway di convergenza *un solo* token. Non appena questo si presenta, il gateway lo fa passare istantaneamente al task successivo, senza mettersi in attesa di token provenienti dagli altri rami (che, per logica XOR, sono vuoti).

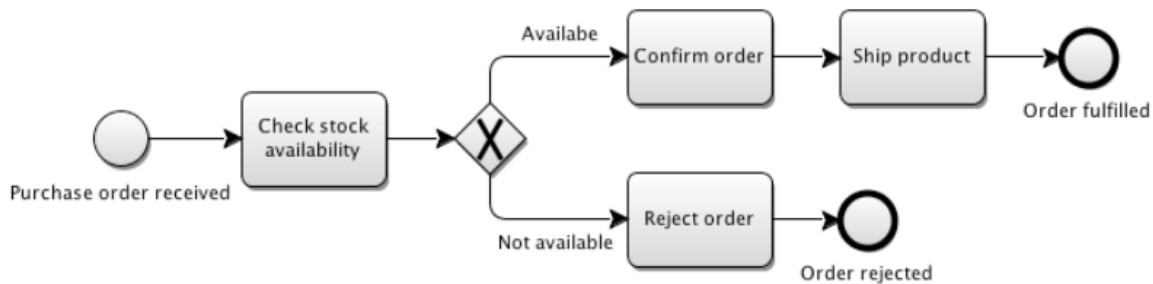
2.9 Esempio Pratico: Spedizione di un Ordine

Per visualizzare l'applicazione del Gateway Esclusivo, analizziamo il processo di spedizione di un ordine (*Order shipment*).

Scenario di Business: Quando viene ricevuto un ordine di acquisto, il processo richiede di verificare la disponibilità del prodotto in magazzino (*check stock availability*). Se l'articolo è disponibile, si procede a confermare l'ordine e a spedire il prodotto; se invece l'articolo non è disponibile (altrimenti), l'ordine viene respinto (*reject order*).

Traduzione in Modello BPMN: Il diagramma traduce fedelmente questo scenario tramite la sintassi formale:

1. Il processo viene innescato dall'Evento di Inizio *"Ordine di acquisto ricevuto"* (*Purchase order received*).
2. Il flusso porta al primo Task logico: *"Verifica disponibilità magazzino"* (*Check stock availability*).
3. Subito dopo interviene il **Gateway Esclusivo Divergente (X)** che valuta l'esito della verifica e divide il percorso in due opzioni mutuamente esclusive:
 - *Ramo 1 (Condizione: "Disponibile")*: Il token percorre questo flusso verso i task in sequenza *"Conferma ordine"* (*Confirm order*) e *"Spedisci prodotto"* (*Ship product*), chiudendo il processo nell'Evento di Fine positivo *"Ordine evaso"* (*Order fulfilled*).
 - *Ramo 2 (Condizione: "Non disponibile")*: Il token percorre questo ramo alternativo verso il task *"Respingi ordine"* (*Reject order*), terminando nell'Evento di Fine negativo *"Ordine respinto"* (*Order rejected*).



Focus sulle Best Practice di Modellazione

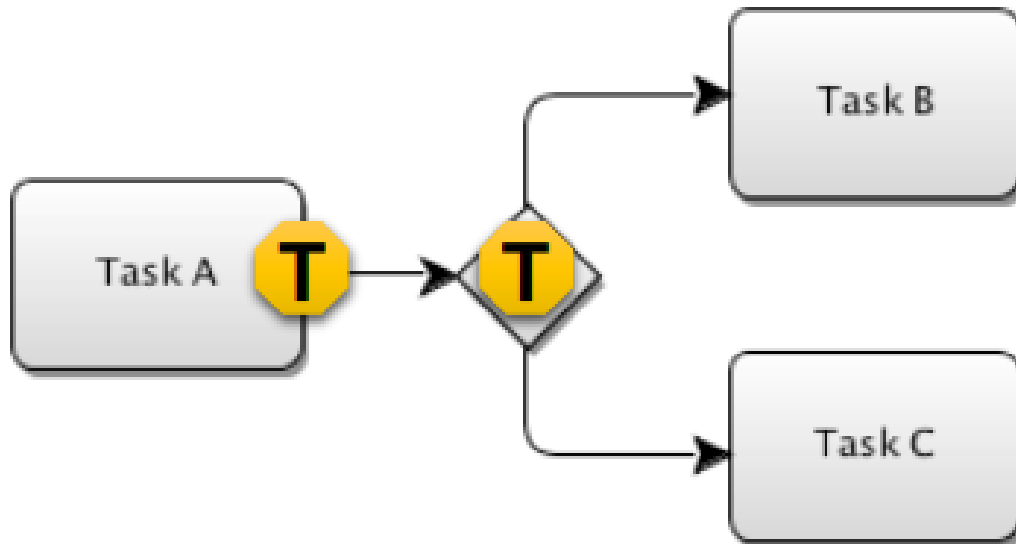
Nell'esempio della Spedizione Ordine, notiamo che l'analista ha inserito una dicitura (etichetta o *label*) direttamente sulle frecce uscenti dal Gateway ("Available" e "Not available"). Questa è una pratica fondamentale. Un Gateway di diramazione rappresenta una domanda (in questo caso sollevata dal task precedente), ma le **condizioni per procedere** devono essere esplicitate sui singoli flussi di sequenza. Un modello privo di queste etichette risulterebbe illeggibile dal punto di vista del business e impossibile da automatizzare per i sistemi informativi.

2.10 Gateway Parallelo (Parallel Gateway - AND)

A differenza del Gateway Esclusivo, il **Gateway Parallelo** (rappresentato da un rombo contenente il simbolo "+") non valuta alcuna condizione o alternativa, ma serve a gestire le attività che devono, o possono, essere eseguite simultaneamente (in parallelo) all'interno del processo.

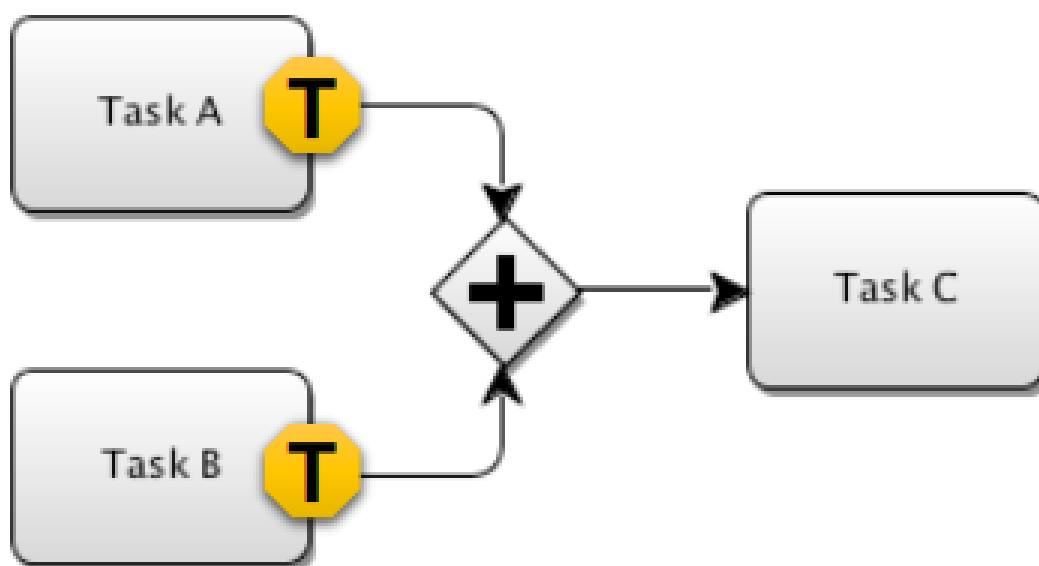
Anche in questo caso, la sua natura è duplice:

Divergenza Parallela (Diverging Parallel Gateway)



- Viene utilizzato per creare percorsi paralleli (*parallel paths*) **senza verificare alcuna condizione**.
- Quando il flusso raggiunge questo gateway, il token in ingresso si "moltiplica": **ogni** flusso di sequenza in uscita riceve un token. In questo modo, tutte le attività sui rami successivi vengono innescate contemporaneamente.

Convergenza Parallela (Converging Parallel Gateway)



- Viene utilizzato per sincronizzare e unire (*combine*) i flussi paralleli creati in precedenza.
- Logica di **Attesa (Waits)**: questo gateway attende l'arrivo dei token da **tutti** i flussi in ingresso prima di instradare un singolo token verso il flusso in uscita. Applica una stretta logica "AND".

Chiarimento Accademico: L'Importanza della Sincronizzazione (AND)

Il concetto di attesa del Gateway Parallelo Convergente è cruciale per l'automazione dei processi. Se un processo si divide in due rami (Es. Ramo 1: "Prepara la merce" / Ramo 2: "Prepara la fattura"), l'attività finale di "Spedizione" non può logicamente iniziare se *entrambe* le attività precedenti non sono concluse. Il gateway convergente "+" funge da "barriera" (Synchronization): blocca il token del ramo che finisce prima, mettendolo in attesa finché non arriva anche il token del ramo più lento. Solo a quel punto, il flusso prosegue.

2.11 Esempio Pratico: Controlli di Sicurezza all'Imbarco

Per visualizzare il funzionamento dei flussi paralleli, analizziamo lo scenario tipico dei controlli aeroportuali.

Scenario di Business: Avendo a disposizione la carta d'imbarco (*boarding pass*), il passeggero si reca ai controlli di sicurezza. Qui subisce il controllo del bagaglio (*luggage screening*) e, parallelamente, il controllo sulla persona (*security screening*). Solo quando ha superato **entrambi** i controlli, può procedere verso l'imbarco (*go to boarding*).

Traduzione in Modello BPMN:

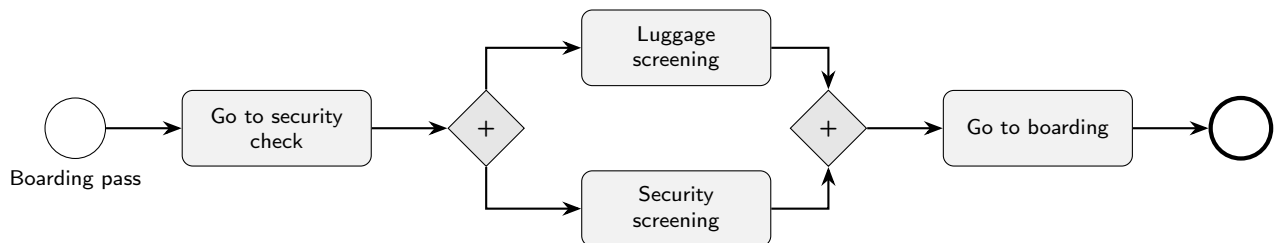


Figura 1: Esempio di Gateway Parallelo (Boarding security check)

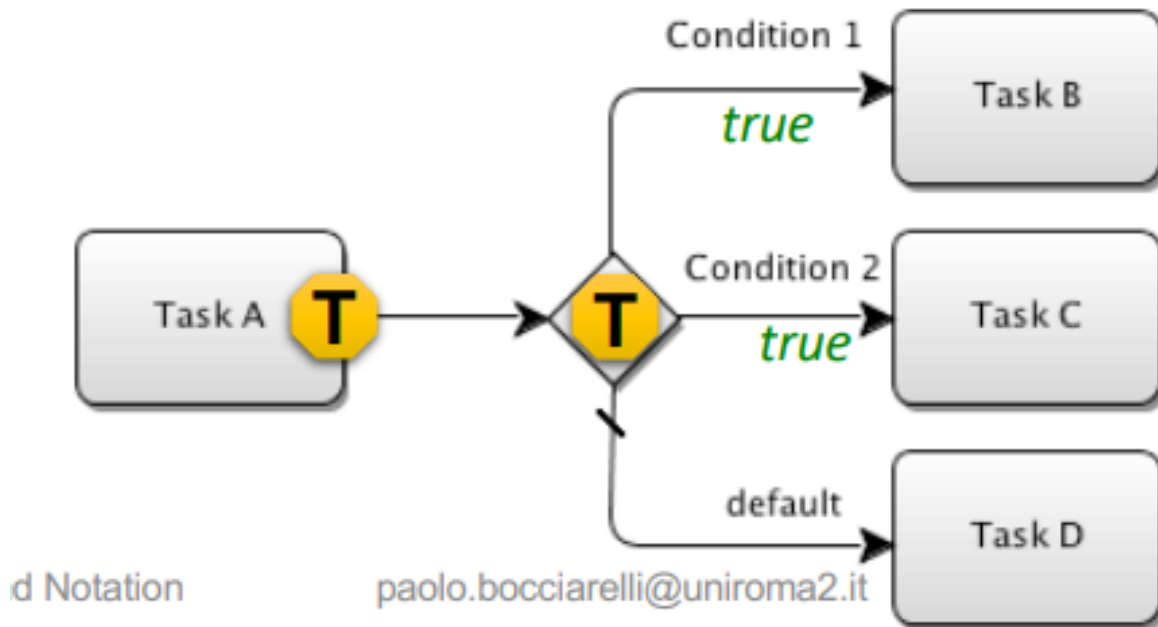
Come mostrato nel diagramma (Figura 1), il processo parte dall'evento di inizio. Dopo la prima attività, il gateway divergente "+" sdoppia il flusso. L'attività di screening del bagaglio e quella della persona non hanno un ordine cronologico prefissato e avvengono in parallelo. Il gateway convergente "+" impedisce al passeggero di procedere all'attività "Go to boarding" finché i flussi di entrambi gli screening non si sono ricongiunti con successo.

2.12 Gateway Inclusivo (Inclusive Gateway - OR)

Il **Gateway Inclusivo**, rappresentato graficamente da un rombo contenente un cerchio al suo interno, combina le caratteristiche dei due gateway precedenti. Permette di ge-

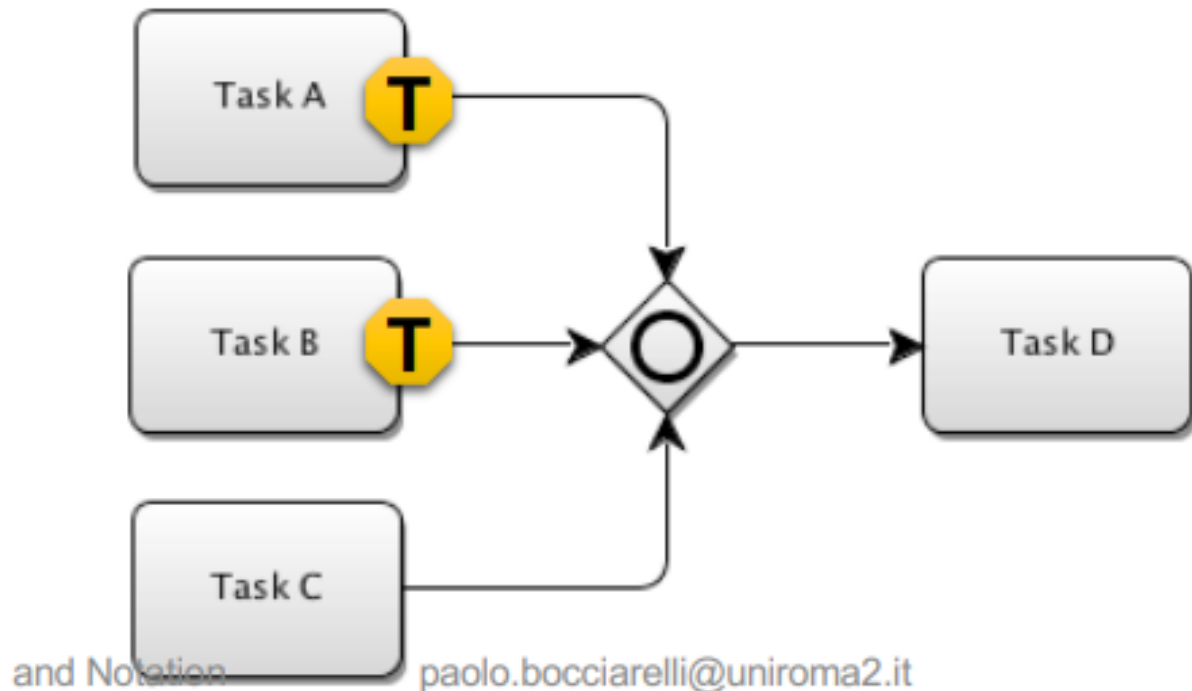
stire scenari in cui più percorsi possono essere attivati contemporaneamente, ma solo al verificarsi di specifiche condizioni.

Divergenza Inclusiva (Diverging Inclusive Gateway)



- Viene utilizzato per creare percorsi che sono alternativi, ma che possono essere **anche paralleli** all'interno del flusso di processo.
- A differenza del Gateway Esclusivo (XOR) in cui si sceglie una sola strada, nel Gateway Inclusivo **tutte le espressioni condizionali vengono valutate**.
- Regola Logica (OR): Il token attraverserà **tutti i flussi di sequenza** per i quali la valutazione della condizione risulta vera (*true*).
- Come per l'XOR, è possibile definire un percorso predefinito (*default path*) che viene intrapreso qualora nessuna delle espressioni condizionali risulti vera.

Convergenza Inclusiva (Converging Inclusive Gateway)



- Viene utilizzato per unire in modo sicuro una combinazione di percorsi alternativi e paralleli.
- Logica di **Attesa Intelligente**: Il token che arriva a un gateway inclusivo convergente può essere sincronizzato con altri token in arrivo successivamente.
- Il token prosegue sul flusso di sequenza in uscita *solo quando tutti i token attesi (expected tokens) sono arrivati*.

Chiarimento Accademico: Il Concetto di "Token Atteso"

La convergenza inclusiva è la più complessa da gestire per i motori di automazione (Workflow Engine). Nel Gateway Parallelo (+), il sistema sa che deve aspettare *sempre tutti* i rami. Nel Gateway Inclusivo (O), i rami attivati a monte possono essere 1, 2, o 5 su 10, a seconda delle condizioni. Il gateway convergente deve quindi "guardare all'indietro" (*look-back*) nel processo per calcolare quanti token sono stati generati dal gateway divergente e mettersi in attesa esattamente di quel numero di token, non uno di più, non uno di meno, prima di sbloccare il flusso.

2.13 Esempio Pratico: Scomposizione dell'Ordine

Per visualizzare il comportamento del Gateway Inclusivo, esaminiamo il processo di scomposizione di un ordine (*Order decomposition*).

Scenario di Business: Quando viene ricevuto un ordine da un cliente, questo può contenere diverse tipologie di merce. La regola impone di: prelevare i prodotti di tipo A

dal Magazzino A (se richiesti nell'ordine) e prelevare i prodotti di tipo B dal Magazzino B (se richiesti nell'ordine).

Se l'ordine contiene solo merce A, si attiverà solo il ramo superiore; se contiene solo merce B, solo il ramo inferiore; se contiene entrambe, si attiveranno entrambi i rami in parallelo.

Traduzione in Modello BPMN:

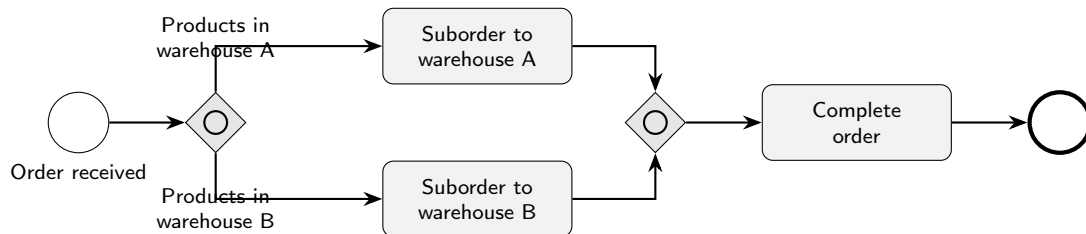


Figura 2: Esempio di Gateway Inclusivo (Order decomposition)

Come illustrato in Figura 2, l'Evento di Inizio "Order received" innesca il Gateway Inclusivo Divergente. Le etichette sui flussi ("*Products in warehouse A*" e "*Products in warehouse B*") fungono da test condizionale. Una volta completate le singole procedure di sottomissione dell'ordine ai magazzini necessari, il Gateway Convergente Inclusivo garantisce che l'attività finale "Complete order" venga eseguita una sola volta e solo dopo che le operazioni logistiche attese si sono concluse con successo.

2.14 Modellazione delle Risorse (Resource Modeling)

Nei diagrammi BPMN, non è sufficiente modellare esclusivamente la sequenza delle attività; è altrettanto essenziale definire *chi* esegue il lavoro. Le risorse organizzative (*organizational resources*) vengono mappate visivamente all'interno dei modelli utilizzando due specifici elementi grafici: le **Pool** e le **Lane**.

Pools (Entità Indipendenti)

- Le Pool rappresentano *entità organizzative indipendenti* (*independent organizational entities*).
- La caratteristica distintiva di due Pool differenti è che esse **non condividono alcun sistema comune** che permetta loro di comunicare in modo implicito (*do not share any common system*).
- *Esempio pratico:* In un processo di acquisto, l'entità "Cliente" (*Customer*) opera in totale indipendenza rispetto all'entità "Fornitore" (*Supplier*). Ciascuno avrà la propria Pool separata.

Lanes (Corsie Operative / Ruoli)

- Le Lane rappresentano *molteplici classi di risorse* (ruoli, reparti o uffici) che operano all'interno della *stessa organizzazione* (*same organization*).

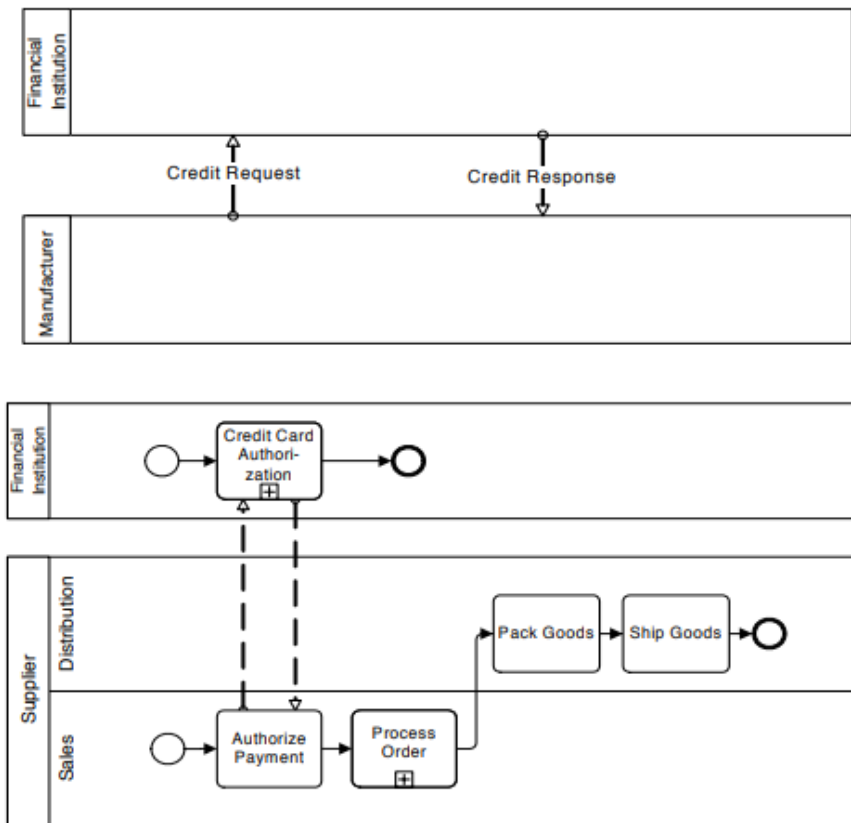
- A differenza delle Pool, le entità collocate in diverse Lane della medesima organizzazione **condividono sistemi e processi comuni** (*share common systems*).
- *Esempio pratico:* All'interno dell'azienda Fornitore (una singola Pool), potremmo avere una Lane per il "Dipartimento Vendite" (*Sales department*) e una Lane separata per il "Dipartimento Marketing" (*Marketing department*).

Chiarimento Accademico: La Regola d'Oro della Comunicazione

La distinzione tra Pool e Lane non è solo visiva, ma impone regole sintattiche rigide nel BPMN:

- **Dentro la stessa Pool (tra le Lane):** Il passaggio di consegne (handover) tra dipartimenti della stessa azienda avviene tramite il normale **Flusso di Sequenza** (freccia a linea continua). Il "Token" passa liberamente da una Lane all'altra.
- **Tra Pool differenti (B2B):** Un Flusso di Sequenza *non può mai* oltrepassare i confini di una Pool. Poiché Cliente e Fornitore non condividono gli stessi sistemi IT, non possono passarsi il Token. La comunicazione tra due Pool avviene **esclusivamente tramite Flussi di Messaggio** (*Message Flows*, rappresentati da frecce tratteggiate), che indicano lo scambio di informazioni esterne (es. l'invio di un'e-mail o di una fattura).

Pools and Lanes



2.15 Flusso di Messaggio (Message Flow)

Il Flusso di Messaggio (rappresentato graficamente da una linea tratteggiata con una punta a freccia vuota) è il costrutto fondamentale per la comunicazione B2B (Business-to-Business) o tra entità completamente indipendenti.

- Viene utilizzato per mostrare il **flusso di messaggi** (*flow of messages*) scambiati tra due partecipanti al processo. Si presuppone che i partecipanti siano predisposti e preparati sia per l'invio che per la ricezione di tali messaggi.
- **Regole di Connessione:** Un flusso di messaggio può connettere **solo due Pool separate**.
- Può essere agganciato visivamente all'intero perimetro della Pool, a specifiche attività (Task) oppure a particolari eventi (Message Events).
- **Vincolo Assoluto:** Un flusso di messaggio **non può mai connettere due oggetti situati all'interno della stessa Pool** (*cannot connect two objects within the same pool*). All'interno della stessa Pool si usa esclusivamente il Flusso di Sequenza.

2.16 Artefatti (Artifacts)

Gli **Artefatti** vengono utilizzati per arricchire il modello fornendo *informazioni aggiuntive* (*additional information*).

La loro caratteristica fondamentale è che **non sono direttamente correlati** alla logica esecutiva primaria del diagramma: non alterano né i flussi di sequenza (*sequence flows*) né i flussi di messaggio (*message flows*).

- Le tre principali tipologie di artefatti previste dallo standard includono:
 - **Associazioni** (*Associations*)
 - **Gruppi** (*Groups*)
 - **Annotazioni testuali** (*Text annotations*)

Associazione (Association)

L'Associazione viene utilizzata specificamente per **connettere** (*associare*) **gli artefatti agli elementi base del processo** (es. collegare una nota testuale a un Task). Graficamente, viene rappresentata da una linea tratteggiata.

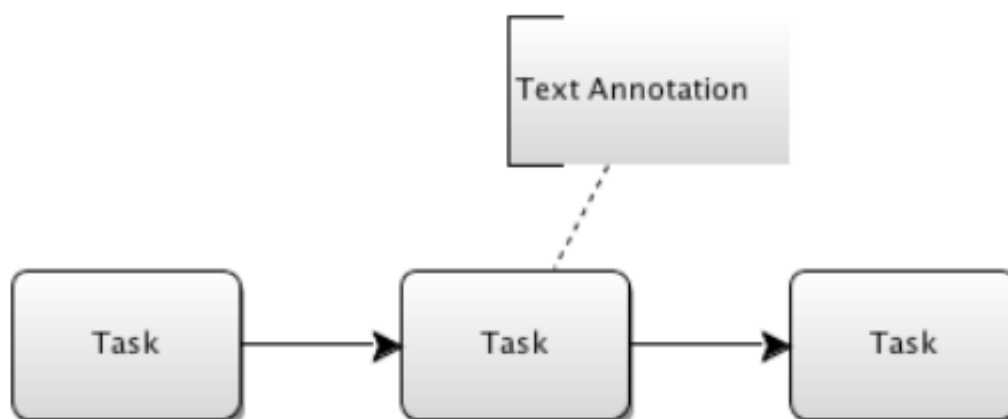
Association

Chiarimento Accademico: L'Artefatto e il Token

Dal punto di vista semantico e dell'automazione, gli artefatti sono completamente "trasparenti". Il *Token* (il gettone esecutivo del processo) non viaggia **mai** lungo una linea di Associazione. Se un Process Designer aggiunge un'annotazione di testo a un'attività per specificare "Usare il modulo 4B", sta fornendo un'informazione fondamentale per la comprensione umana (es. per il lavoratore che legge il diagramma), ma il motore software (Workflow Engine) ignorerà totalmente quell'artefatto durante l'esecuzione del codice.

Annotazione Testuale (Text Annotation)

Le Annotazioni Testuali rappresentano il metodo più diretto e leggibile per integrare documentazione all'interno del modello visivo.



- Il loro scopo esclusivo è quello di *fornire informazioni aggiuntive* (*additional information*) pensate appositamente per il lettore umano del diagramma BPMN.
- Come tutti gli artefatti, la loro presenza *non influisce in alcun modo sul flusso del processo* (*does not affect the flow*) o sull'automazione dello stesso.

Text Annotation
(Informazioni per il
lettore)

Focus sulle Best Practice: La Documentazione Integrata

Sebbene non abbiano un peso "informatico", le annotazioni testuali sono di vitale importanza nella fase di "Process Discovery". Spesso un Task come "Valuta Richiesta" nasconde una logica umana complessa. Utilizzare un'annotazione testuale per scrivere "La valutazione si basa sulle linee guida del manuale operativo v2.1, capitolo 4" trasforma un semplice diagramma grafico in un vero e proprio **strumento di knowledge management** (gestione della conoscenza) per i neo-assunti o gli analisti futuri, prevenendo la perdita del know-how aziendale.

2.17 Oggetti Dati (Data Objects)

Nei processi aziendali reali, l'esecuzione delle attività richiede spesso la lettura o la produzione di documenti e informazioni. Gli **Oggetti Dati** vengono utilizzati per rappresentare esplicitamente i dati e i documenti che vengono scambiati durante l'esecuzione del processo.

- Il loro scopo principale è mostrare visivamente l'*input* e l'*output* necessari o prodotti dalle specifiche attività.
- Lo standard BPMN definisce formalmente **5 diverse tipologie** di oggetti dati, ciascuna con un proprio simbolo grafico dedicato.

Le 5 Tipologie di Oggetti Dati

- **Oggetto Dati (*Data Object*):** Rappresenta le informazioni standard che attraversano il processo (es. e-mail, lettere, documenti contrattuali).
- **Oggetto Dati di Raccolta (*Collection Data Object*):** Rappresenta una raccolta, o un insieme multiplo, di informazioni dello stesso tipo (es. una lista della spesa o un batch di fatture).
- **Input di Dati (*Data Input*):** Rappresenta un input esterno che è strettamente necessario per innescare o avviare un'attività.
- **Output di Dati (*Data Output*):** Rappresenta l'esito formale, o il risultato generato al termine di una singola attività o dell'intero processo.
- **Archivio Dati (*Data Store*):** Rappresenta un luogo (tipicamente un database o un archivio fisico) in cui il processo può leggere e scrivere dati in modo strutturato.

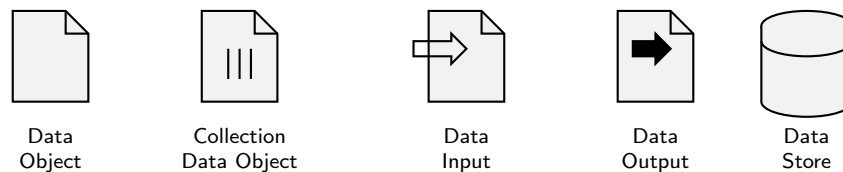


Figura 3: I 5 simboli degli Oggetti Dati nel BPMN

Chiarimento Accademico: Dati Temporanei vs. Persistenti

In ambito di automazione e sistemi IT, vi è una differenza ontologica fondamentale tra i primi quattro simboli e l'ultimo (*Data Store*).

- I **Data Object** (e le relative varianti Collection, Input e Output) sono dati **temporanei** (*transient data*). Esistono e vivono esclusivamente per la durata della singola esecuzione del processo (la singola istanza). Quando l'ordine viene completato, il "modulo compilato in memoria" svanisce logicamente con esso.
- Il **Data Store** rappresenta invece la **persistenza** (*persistent data*). Un database clienti esiste prima che il processo inizi e continuerà a esistere dopo che il processo sarà terminato. Le attività interagiscono con il Data Store (es. query SQL per recuperare l'indirizzo del cliente) per estrarre informazioni strutturate o salvarle in modo permanente a fine processo.

2.18 Tipologie di Processi BPMN (BPMN Process Types)

Quando si analizzano i processi aziendali, lo standard BPMN permette di modellarli secondo tre approcci o tipologie principali, a seconda del livello di interazione e del perimetro organizzativo coinvolto:

1. **Orchestrazione** (*Orchestration*)
2. **Collaborazione** (*Collaborations*)
3. **Coreografia** (*Choreographies*)

- Nello specifico, il linguaggio BPMN utilizza i termini formali *collaborazione* e *coreografia* quando è necessario modellare l'interazione vera e propria tra processi differenti.

1. Orchestrazione (Orchestration)

L'Orchestrazione modella un processo aziendale (BP) che è **strettamente interno** a una specifica organizzazione.

- Poiché non vi sono interazioni con attori esterni autonomi, un processo di orchestrazione viene generalmente chiamato **workflow** o "processo BPM".

- Visivamente, un'orchestrazione è sempre contenuta all'interno di un'unica **Pool** (che può, ovviamente, essere suddivisa in diverse Lane per i dipartimenti interni).

Esempio di Orchestrazione: Di seguito è riportato un classico esempio di orchestrazione interna per l'approvazione di una pratica assicurativa. Il flusso è completamente sequenziale e governato da un'unica entità centrale.

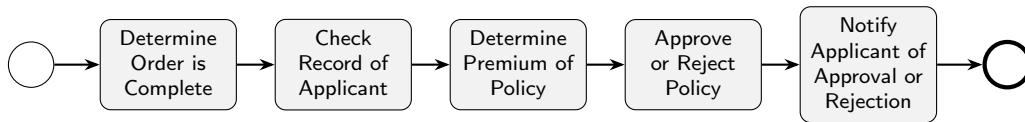


Figura 4: Esempio di processo di Orchestrazione interna

2. Collaborazione (Collaboration)

La Collaborazione è la tipologia di diagramma che mostra in modo esplicito le *interazioni tra due o più partecipanti* (participants) autonomi e distinti.

- Dal punto di vista della modellazione visiva, *una Pool rappresenta uno dei partecipanti* coinvolti nella collaborazione.
- Una Collaborazione può essere descritta come l'insieme di due o più processi (ciascuno orchestrato nella propria Pool) che *comunicano l'uno con l'altro*.
- Il veicolo di questa comunicazione, ovvero il messaggio effettivamente scambiato tra i partecipanti, viene mostrato sul diagramma per mezzo di un *Flusso di Messaggio* (Message Flow).

Collaboration Example

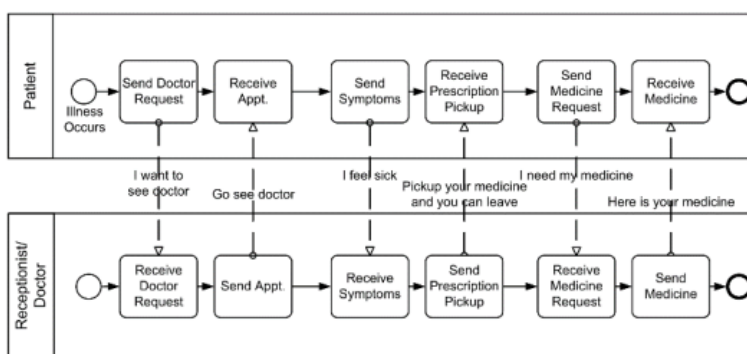
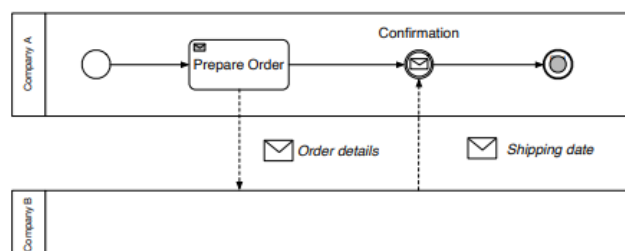


Image from "Business Process Model and Notation (BPMN)", <http://www.omg.org/spec/BPMN/2.0>



Chiarimento Accademico: Orchestrazione vs. Collaborazione

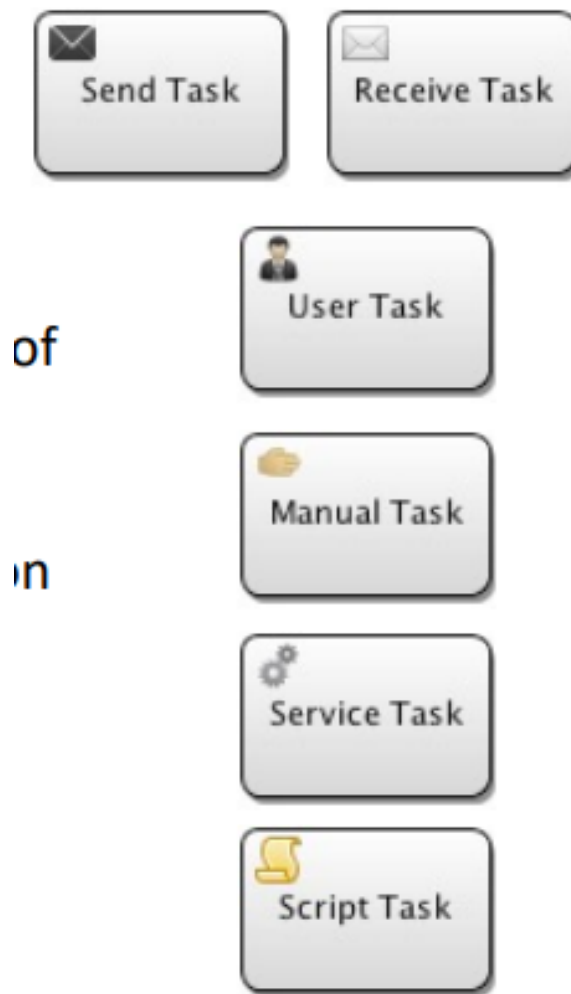
Per unire i concetti visti finora, possiamo affermare che la **Collaborazione** è di fatto formata da due o più **Orchestrazioni** interconnesse. Riprendendo l'esempio del B2B precedente: il Cliente "orchestra" il proprio processo di acquisto nella sua Pool; il Fornitore "orchestra" il proprio processo di evasione ordine nella sua Pool. Ciascuno è padrone in casa propria. Il diagramma globale che li unisce tramite le frecce tratteggiate (i messaggi scambiati, come Ordine e Fattura) prende il nome tecnico di **Diagramma di Collaborazione**.

2.19 Task Specializzati (Specialized Tasks)

Fino a questo momento abbiamo utilizzato il simbolo del Task generico (un semplice rettangolo con angoli smussati). Tuttavia, lo standard BPMN mette a disposizione diverse tipologie di **Task Specializzati** per definire con maggiore precisione e dettaglio la natura tecnica e operativa dell'attività da svolgere.

Visivamente, i task specializzati si distinguono per la presenza di un **marcatore (icona)** nell'angolo in alto a sinistra del rettangolo. Le principali tipologie includono:

- **Invio / Ricezione di Messaggi (*Send/Receive message*):**
 - *Send Task (Busta nera)*: Un task dedicato esclusivamente all'invio di un messaggio verso un partecipante esterno (all'interno delle logiche del processo).
 - *Receive Task (Busta bianca)*: Un task che rimane in attesa di ricevere un messaggio da un partecipante esterno prima di potersi dichiarare concluso.
- **Attività Utente (*User activity*):** Un task eseguito da un operatore umano con l'assistenza e il supporto di un'applicazione software.
- **Attività Manuale (*Manual activity*):** Un task eseguito fisicamente da un operatore umano senza alcun ausilio di applicazioni software o sistemi informatici.
- **Chiamata a Servizio (*Service calling*):** Un task automatizzato che utilizza una qualche forma di servizio esterno (es. un Web Service), tipicamente identificato tramite un URI (Uniform Resource Identifier).
- **Script (*Script*):** Un task eseguito in totale autonomia dal motore di esecuzione dei processi aziendali (*business process engine*), generalmente eseguendo una porzione di codice interno.



2.20 Definizione degli Eventi (Event Definition)

Fino ad ora abbiamo visto gli eventi nella loro forma più semplice (Inizio e Fine). Tuttavia, nello standard BPMN, un "Evento" è un costrutto complesso la cui semantica è determinata dalla combinazione (composizione) di **quattro caratteristiche fondamentali**:

1. **Posizione (*Position*)**: Determina la collocazione topologica dell'evento all'interno del flusso di processo. Può essere:
 - *Inizio (Start)*: Avvia il processo.
 - *Intermedio (Intermediate)*: Si verifica durante l'esecuzione del processo.
 - *Fine (End)*: Conclude il processo o un suo ramo.
2. **Natura (*Nature*)**: Definisce il comportamento attivo o passivo dell'evento rispetto al verificarsi della condizione. Si divide in:
 - *Catch (Cattura / Ricezione)*: L'evento attende che si verifichi un determinato stimolo.

- *Throw (Lancio / Generazione)*: L'evento genera attivamente uno stimolo.
3. **Impatto (*Impact*)**: Indica in che modo l'evento condiziona l'attività o il sottoprocesso a cui è eventualmente associato. Può essere:
- *Interrompente (Interrupting)*: Ferma e annulla l'attività in corso.
 - *Non-Interrompente (Non-Interrupting)*: Avvia un flusso parallelo senza fermare l'attività principale.
4. **Tipo (*Type*)**: Specifica la causa materiale o la condizione logica che caratterizza l'evento. Alcuni esempi comuni sono:
- *Messaggio (Message), Timer, Errore, Segnale, ecc..*

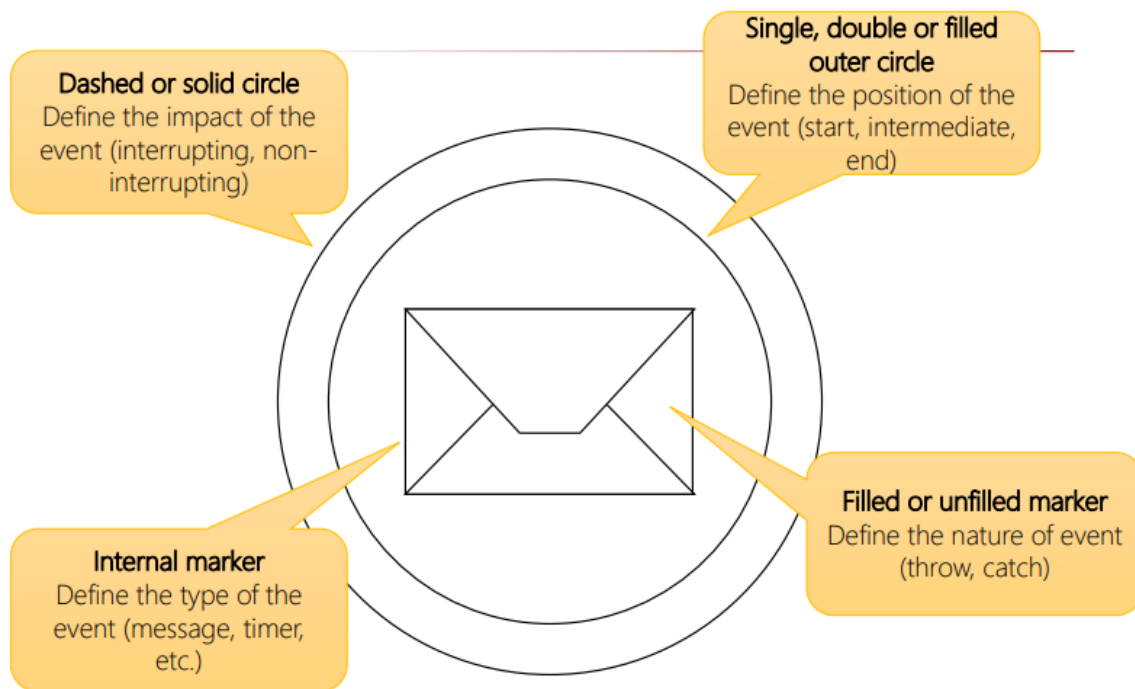
Chiarimento Accademico: Natura Catch vs. Throw

La distinzione tra **Catch** e **Throw** è vitale per comprendere come si muove il *Token* (il gettone di esecuzione) all'interno del motore software:

- **Catch (Semaforo Rosso)**: Quando il Token raggiunge un evento Intermedio di tipo Catch (es. "Attesa ricezione documenti"), **si ferma**. Il sistema entra in uno stato di attesa passiva finché il trigger esterno non "sblocca" l'evento, permettendo al Token di proseguire.
- **Throw (Azione Istantanea)**: Quando il Token raggiunge un evento Intermedio di tipo Throw (es. "Invia notifica di ritardo"), l'evento scatta istantaneamente lanciando il suo trigger verso l'esterno. Il Token **non si ferma**, ma lo attraversa immediatamente proseguendo verso il Task successivo.

Nota logica: Per definizione, gli Eventi di Inizio possono essere solo di tipo Catch (reagiscono a un innesco per far partire il Token), mentre gli Eventi di Fine possono essere solo di tipo Throw (consumano il Token e generano un risultato finale).

Event Symbol



Posizione dell'Evento (Event Position)

La prima dimensione che caratterizza un evento è la sua posizione all'interno del ciclo di vita del processo. Il BPMN utilizza lo spessore e il numero delle linee di bordo del cerchio per comunicare visivamente questa informazione:

- **Evento di Inizio (*Start event*)**: Rappresenta il punto iniziale assoluto del flusso di esecuzione (*initial point of execution flow*). Graficamente è delineato da un **singolo bordo sottile**.
- **Evento Intermedio (*Intermediate event*)**: Rappresenta un evento che può verificarsi in un momento qualsiasi compreso tra l'inizio e la fine del flusso di esecuzione (*may occur between start and end*). Graficamente è delineato da un **doppio bordo sottile**.
- **Evento di Fine (*End event*)**: Rappresenta il punto di arresto definitivo del flusso di esecuzione (*stop of the execution flow*). Graficamente è delineato da un **singolo bordo spesso** (in grassetto).

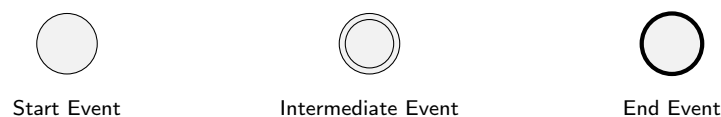


Figura 5: Notazione grafica della Posizione dell'Evento nel BPMN

Chiarimento Accademico: Il Linguaggio Visivo del BPMN

Il BPMN è stato progettato per essere letto intuitivamente anche senza conoscere a memoria il manuale. La logica dei bordi ne è un esempio perfetto:

- Il **bordo sottile singolo** dell'Evento di Inizio suggerisce un punto di ingresso "leggero", pronto ad accogliere uno stimolo esterno per far partire il processo.
- Il **doppio cerchio** dell'Evento Intermedio ricorda l'icona di un "target" (bersaglio) o di un ingranaggio: indica un punto focale durante la lavorazione, dove il processo deve temporaneamente concentrarsi o generare un'azione specifica.
- Il **bordo spesso e pesante** dell'Evento di Fine funge visivamente da "muro": indica in modo inequivocabile che il *Token* non può andare oltre e viene definitivamente assorbito dal sistema.

Natura dell'Evento (Event Nature)

La seconda dimensione caratterizzante riguarda il modo in cui l'evento interagisce con il suo trigger (lo stimolo logico o fisico). Nello standard BPMN si distinguono due nature diametralmente opposte:

- **Evento di Ricezione (*Catch event*):**
 - **Tutti** gli eventi di Inizio e **alcuni** eventi Intermedi appartengono a questa categoria.
 - La regola generale stabilisce che, nel momento in cui l'evento viene innescato (*triggered*) dal verificarsi della condizione attesa, ***viene generato il token*** all'interno del flusso di processo.
- **Evento di Lancio (*Throw event*):**
 - **Tutti** gli eventi di Fine e **alcuni** eventi Intermedi agiscono come eventi di lancio (*throwing events*), i quali generano attivamente un trigger che potrebbe eventualmente essere catturato in seguito da un altro evento.
 - Tipicamente, il trigger ***trasporta delle informazioni***, trasferendole dall'ambito (*scope*) in cui si è verificato l'evento di lancio verso l'ambito in cui risiede l'evento di ricezione (*catching event*).

Events Definition

Events

	Standard	Start		Intermediate			End	
		Event Sub-Process Interrupting	Event Sub-Process Non-Interrupting	Catching	Boundary Interrupting	Boundary Non-Interrupting	Throwing	Standard
None: Untyped events, indicate start point, state changes or final states.								
Message: Receiving and sending messages.								
Timer: Cyclic timer events, points in time, time spans or timeouts.								
Escalation: Escalating to an higher level of responsibility.								

Events Definition

Events

	Start			Intermediate			End
	Standard	Event Sub-Process Interrupting	Event Sub-Process Non-Interrupting	Catching	Boundary Interrupting	Boundary Non-Interrupting	Throwing
Compensation: Handling or triggering compensation.							
Signal: Signalling across different processes. A signal thrown can be caught multiple times.							
Multiple: Catching one out of a set of events. Throwing all events defined							
Parallel Multiple: Catching all out of a set of parallel events.							
Terminate: Triggering the immediate termination of a process.							

Business Process Model and Notation

paolo.bocciarelli@uniroma2.it

Events Definition

Events	Start			Intermediate			End
	Standard	Event Sub-Process Interrupting	Event Sub-Process Non-Interrupting	Catching	Boundary Interrupting	Boundary Non-Interrupting	Throwing
Conditional: Reacting to changed business conditions or integrating business rules.							
Link: Off-page connectors. Two corresponding link events equal a sequence flow.							
Error: Catching or throwing named errors.							
Cancel: Reacting to cancelled transactions or triggering cancellation.							

2.21 Gateway Basato sugli Eventi (Event-Based Gateway)

Mentre i gateway visti in precedenza (come l'Esclusivo o l'Inclusivo) instradano il flusso basandosi sulla valutazione di dati o espressioni logiche (es. "Importo > 1000€"), il **Gateway Basato sugli Eventi** utilizza un approccio radicalmente diverso.

- La scelta dei percorsi alternativi si basa *esclusivamente sugli eventi che si verificano* nel mondo reale o nel sistema, piuttosto che sulla valutazione di espressioni predefinite.
- Tipicamente, è la *ricezione di un messaggio* (da parte di un attore esterno) a determinare fisicamente quale percorso verrà intrapreso.

La Condizione di Gara (Race Condition)

Il meccanismo di funzionamento di questo gateway segue una stretta logica competitiva:

- Nel momento in cui il *primo evento* tra quelli previsti si innesca (viene "triggerato"), il percorso ad esso corrispondente viene immediatamente attivato.
- Di conseguenza, *tutti i restanti percorsi perdono la loro validità* e vengono annullati (i token in attesa sugli altri eventi vengono distrutti).

- Il gateway basato sugli eventi rappresenta a tutti gli effetti una condizione di gara (*race condition*), in cui il primo evento che si verifica "vince" e prende il controllo del flusso.

Istanziamento del Processo

Oltre a essere utilizzato per dividere il flusso a metà di un processo esistente, i gateway basati sugli eventi possono essere posizionati all'inizio del diagramma: possono **avviare l'intero processo** indirizzandolo immediatamente verso percorsi diversi a seconda dell'evento scatenante che si è verificato.

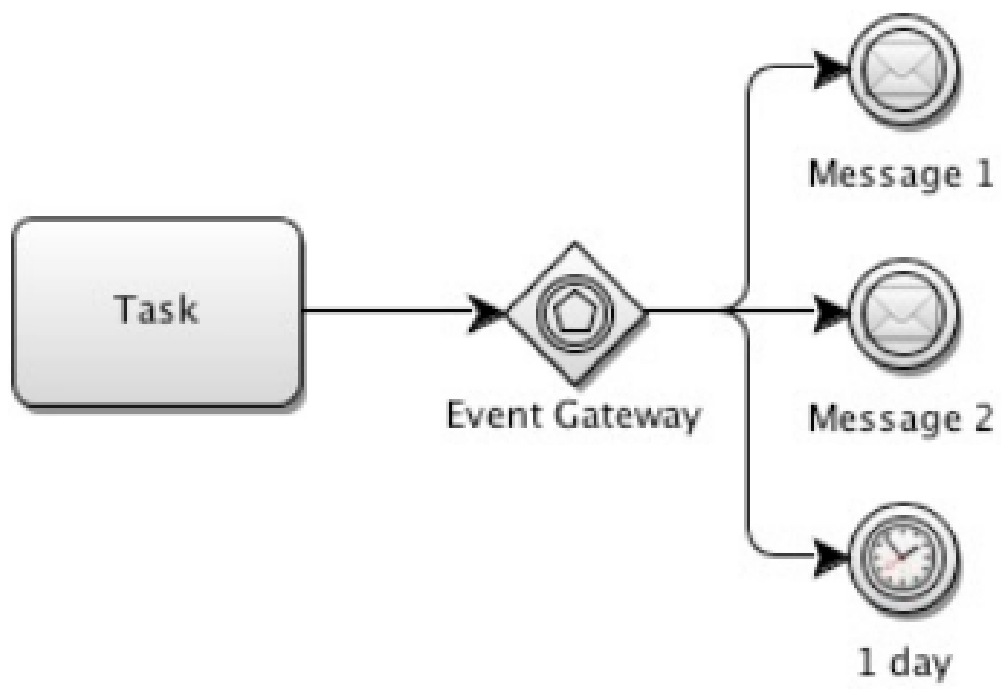


Event-Based
Gateway

Figura 6: Il simbolo del Gateway Basato sugli Eventi nel BPMN

Chiarimento Accademico: Il Paradigma Reattivo

Il Gateway Basato sugli Eventi introduce un paradigma "reattivo" nell'automazione. Mentre un Gateway XOR standard guarda al *passato* (es. valuta un modulo già compilato), l'Event-Based Gateway guarda al *futuro*. Immagina un processo in cui, dopo aver inviato un preventivo, l'azienda debba attendere. Il Token entra nell'Event-Based Gateway e si ferma, mettendosi in attesa su due eventi paralleli: 1) "Ricezione Bonifico" (Catch Message Event) e 2) "Scadenza 30 giorni" (Timer Event). È una gara: se il cliente paga al 29° giorno, il ramo 1 "vince", il processo prosegue verso la spedizione e il timer viene cancellato. Se si arriva al 30° giorno senza bonifico, il Timer scatta per primo vincendo la gara, e il processo prosegue verso il ramo dell'annullamento ordine.



3 Lezione 3: Metamodeling & Model Transformations

3.1 Il Ruolo della Modellazione e il Principio di Sostituibilità Limitata

La modellazione è un'attività intrinsecamente legata alla natura umana. Si può affermare che sia essenziale per l'attività umana poiché ogni nostra azione è preceduta dalla costruzione (che essa sia implicita nella nostra mente o esplicita su carta) di un modello.

Ma cosa significano esattamente questi termini in ambito ingegneristico?

- **Che cos'è un sistema (*System*)?** In breve, un sistema è un insieme di elementi che interagiscono tra loro.
- **Che cos'è un modello (*Model*)?** Un modello è una rappresentazione *astratta* (ovvero semplificata) di un sistema.

Il Principio di Sostituibilità (Principle of Substitutability)

Il rapporto formale che lega un Modello al Sistema reale che esso descrive è governato da una precisa definizione logica:

Si dice che un modello M è una rappresentazione di un sistema S per un dato insieme di domande Q se, per ogni singola domanda appartenente a questo insieme Q , **il modello M fornirà esattamente la stessa risposta che il sistema S avrebbe fornito rispondendo alla medesima domanda.**

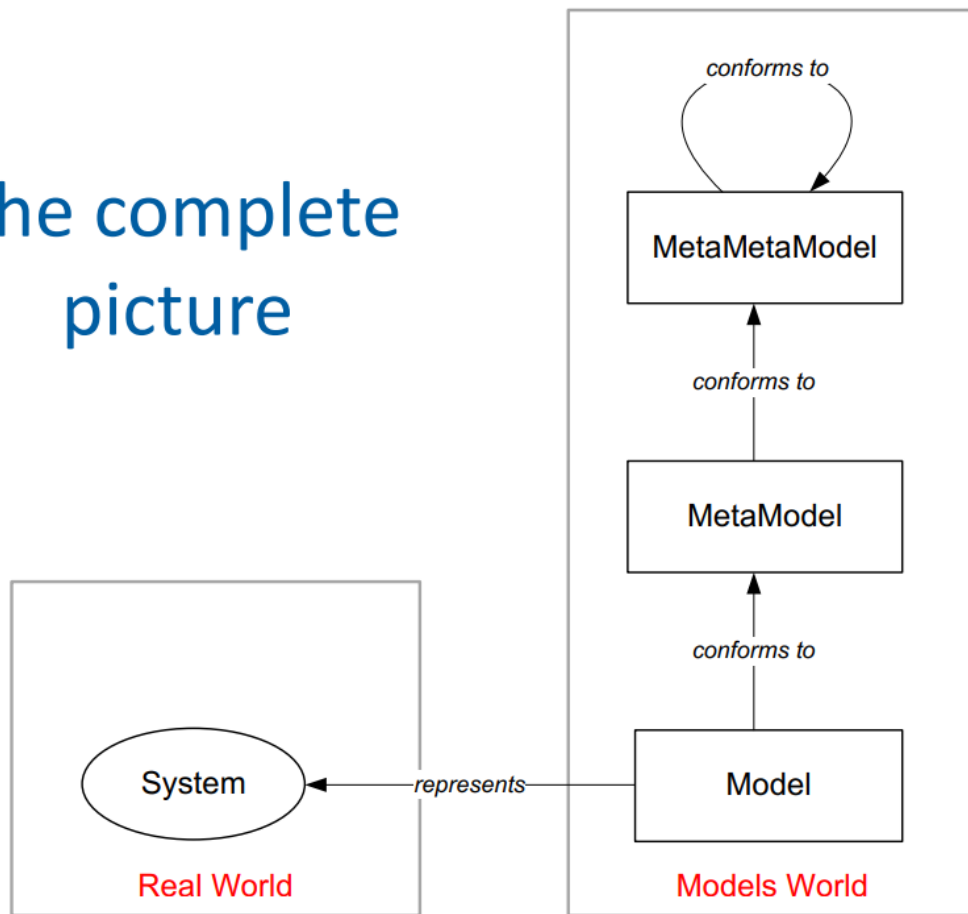


Figura 7: Relazione logica tra Sistema e Modello

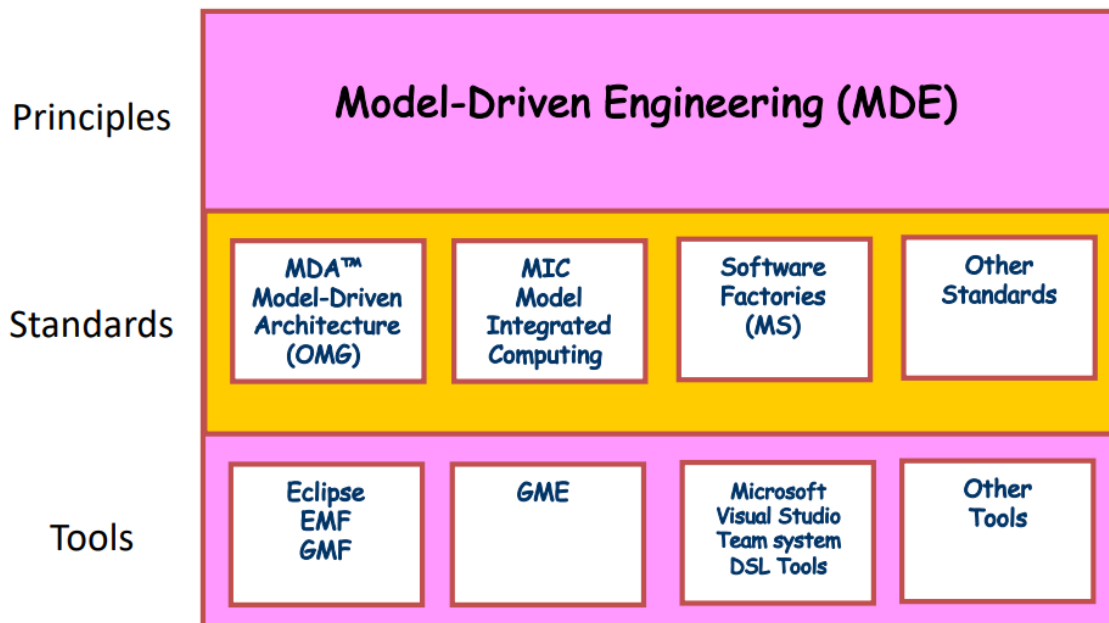
Chiarimento Accademico: Perché Sostituibilità "Limitata"?

Il principio si definisce a sostituibilità *limitata* per via della clausola "per un dato insieme di domande Q ". Nessun modello è una replica perfetta e onnicomprensiva della realtà, altrimenti non sarebbe un'astrazione (sarebbe un clone). Se il Modello M è una mappa stradale di Roma (il Sistema S), alla domanda Q_1 ("Qual è la via più breve per il Colosseo?") la mappa darà la stessa risposta della realtà. In questo senso, la mappa *sostituisce* validamente la città. Tuttavia, alla domanda Q_2 ("Piove a Roma oggi?"), la mappa stradale non sa rispondere. Per questo insieme di domande, il modello perde la sua sostituibilità. L'ingegnere deve sempre assicurarsi che il modello creato sia valido per l'insieme di domande (Q) a cui il progetto deve effettivamente rispondere.

The complete picture



Principles, Standards and Tools



3.2 Model-Driven Architecture (MDA) di OMG

L'Object Management Group (OMG) ha definito una propria proposta completa e strutturata per l'applicazione delle pratiche MDE (*Model-Driven Engineering*) allo sviluppo dei sistemi. Il risultato di questo importante sforzo di standardizzazione è la **Model-Driven Architecture (MDA)**.

I concetti cardine su cui si fonda l'architettura MDA sono:

- **Motivazione (*MDA Motivation*):** L'obiettivo primario è trasferire il focus del lavoro ingegneristico dalla pura scrittura del codice (il cui paradigma fondante è "*tutto è un oggetto*") alla progettazione di alto livello (il cui nuovo paradigma diventa "*tutto è un modello*").
- **Linee Guida:** La MDA fornisce un insieme rigoroso di *linee guida* (*guidelines*) per strutturare le specifiche di sistema espresse sotto forma di modelli e per definire e gestire le trasformazioni tra tali modelli.
- **Trasformazioni di Modello (*Model Transformations*):** Una trasformazione consiste nel mappare (tradurre logicamente) gli elementi di un modello sorgente (*source model*) negli elementi di un altro modello, detto modello di destinazione (*target model*).
 - Il modello sorgente è sempre conforme a uno specifico *metamodello* di partenza.
 - Il modello di destinazione generato può essere conforme allo stesso metamodello (es. per rifinire o ottimizzare il modello) oppure a un *metamodello* *diverente*.

Chiarimento Accademico e Collegamento al Progetto

Il passaggio dal motto "tutto è un oggetto" (tipico della programmazione Object-Oriented come Java) a "tutto è un modello" rappresenta la vera rivoluzione dell'ingegneria del software moderna. Invece di scrivere manualmente migliaia di righe di codice, soggette a errori umani e dipendenti da una specifica tecnologia, l'ingegnere progetta un Modello astratto.

Il concetto di **Trasformazione** descritto in questa slide è **esattamente ciò che viene richiesto nel Task 4 del tuo progetto sul Satellite**. Dovrai prendere un modello sorgente conforme al metamodello UML (il tuo diagramma delle classi) e applicare una regola di trasformazione (il motore Acceleo). Questo mapperà gli elementi UML verso un modello di destinazione conforme a un metamodello diverso (il codice testuale `.java`). La MDA serve proprio ad automatizzare questa generazione, garantendo che il codice sia sempre coerente e tracciabile rispetto ai requisiti di partenza.

Panoramica sulla MDA (MDA Overview)

La Model-Driven Architecture si fonda su un'idea ben nota e consolidata nell'ingegneria del software: *separare la specifica* del funzionamento di un sistema *dai dettagli* del modo in cui quel sistema utilizza le funzionalità *della sua piattaforma* di esecuzione.

La MDA fornisce un approccio metodologico (e abilita la creazione di strumenti di supporto) per:

- Specificare un sistema in modo indipendente dalla piattaforma che lo supporta.
- Specificare le piattaforme stesse.
- Scegliere una piattaforma particolare per il sistema.
- **Trasformare** la specifica del sistema in una specifica concreta per la piattaforma selezionata.

I tre obiettivi primari della MDA, raggiunti attraverso questa separazione architetturale degli interessi (*separation of concerns*), sono:

1. **Portabilità** (*Portability*)
2. **Interoperabilità** (*Interoperability*)
3. **Riutilizzabilità** (*Reusability*)

Chiarimento Accademico: Separation of Concerns

Il concetto di "separazione degli interessi" è vitale. Se si progetta un sistema inserendo fin da subito i dettagli del linguaggio Java o del database SQL, il modello sarà "inquinato" e strettamente legato a quelle specifiche tecnologie. Seguendo la MDA, l'ingegnere progetta prima la "logica pura" (il comportamento di business). Solo successivamente, in modo automatico tramite gli strumenti di trasformazione, il modello viene tradotto per una specifica piattaforma. Questo garantisce **riutilizzabilità** e **portabilità**: se in futuro l'azienda vorrà migrare da Java a un nuovo linguaggio, non dovrà riprogettare il sistema da zero; basterà applicare una nuova regola di trasformazione al modello originale.

I Punti di Vista della MDA (MDA Viewpoints)

Per gestire la complessità dei sistemi software e implementare la separazione degli interessi, la Model-Driven Architecture articola la progettazione in tre differenti punti di vista (*viewpoints*):

- **Punto di Vista Indipendente dal Calcolo** (*Computation Independent Viewpoint*):
 - Questo punto di vista si concentra sull'ambiente in cui opera il sistema e sui requisiti del sistema stesso.
 - In questa prospettiva, i dettagli relativi alla struttura e all'elaborazione interna del sistema sono volutamente nascosti o non ancora determinati.
- **Punto di Vista Indipendente dalla Piattaforma** (*Platform Independent Viewpoint*):
 - Questo punto di vista si concentra sul funzionamento di un sistema, nascondendo al contempo i dettagli tecnici necessari per una particolare piattaforma di esecuzione.

- Una vista indipendente dalla piattaforma mostra esclusivamente quella parte della specifica completa che non cambia passando da una piattaforma all'altra.
- Per realizzarla, si può utilizzare un linguaggio di modellazione di uso generale (*general purpose modeling language*), oppure un linguaggio specifico per il dominio in cui il sistema verrà impiegato.

- **Punto di Vista Specifico della Piattaforma (*Platform Specific Viewpoint*):**

- Questo punto di vista combina la vista indipendente dalla piattaforma con un focus aggiuntivo, rivolto in modo specifico ai dettagli sull'utilizzo di una determinata piattaforma da parte del sistema.

Dai Punti di Vista ai Modelli (CIM)

Nella prassi ingegneristica della Model-Driven Architecture, a ciascuno di questi tre punti di vista teorici corrisponde un artefatto concreto (un Modello) da produrre:

1. Al *Computation Independent Viewpoint* corrisponde il **CIM (Computation Independent Model)**. Spesso coincide con l'analisi del dominio: i diagrammi BPMN che abbiamo studiato nelle lezioni precedenti sono perfetti esempi di CIM. Definiscono "cosa" serve all'azienda senza parlare di software.
2. Al *Platform Independent Viewpoint* corrisponde il **PIM (Platform Independent Model)**. È la logica pura dell'applicazione (es. un diagramma delle Classi UML senza riferimenti a database o framework).
3. Al *Platform Specific Viewpoint* corrisponde il **PSM (Platform Specific Model)**. È il modello pronto per essere compilato in codice (es. un diagramma UML arricchito con stereotipi Java o SQL).

Il cuore dell'approccio Model-Driven è la creazione di *trasformazioni automatiche* per far evolvere il progetto passando fluidamente da un CIM, a un PIM, e infine a un PSM.

3.3 Modello di Riferimento ODP (Open Distributed Processing)

Oltre ai tre punti di vista definiti dalla MDA, nell'ingegneria del software è fondamentale conoscere il **modello di riferimento ODP** (*Open Distributed Processing Reference Model*). In questo framework metodologico, un sistema complesso viene descritto e analizzato attraverso **cinque punti di vista complementari**:

- **Punto di Vista Aziendale (*Enterprise Viewpoint*):** Si concentra sullo scopo, sull'ambito (*scope*) e sulle policy del sistema all'interno del suo specifico contesto organizzativo.
 - *Focus primario*: **Perché** il sistema esiste e **cosa** ci si aspetta che realizzi.
- **Punto di Vista dell'Informazione (*Information Viewpoint*):** Si occupa di gestire la semantica delle informazioni e le modalità di elaborazione delle stesse.

- *Focus primario*: **Quali** informazioni gestisce il sistema.
- **Punto di Vista Computazionale (*Computational Viewpoint*)**: Descrive la scomposizione funzionale del sistema nei suoi elementi logici.
 - *Focus primario*: **Cosa** fa tecnicamente il sistema.
- **Punto di Vista Ingegneristico (*Engineering Viewpoint*)**: Si concentra sui meccanismi e sull'infrastruttura necessari per supportare la distribuzione logica e fisica.
 - *Focus primario*: **Come** il sistema distribuito supporta le interazioni tra i vari componenti.
- **Punto di Vista Tecnologico (*Technology Viewpoint*)**: Riguarda le scelte concrete, materiali e definitive di implementazione.
 - *Focus primario*: **Quali** tecnologie (linguaggi, database, hardware) vengono utilizzate per implementare il sistema.

Chiarimento Accademico: ODP rispetto alla MDA

Il modello ODP e la Model-Driven Architecture (MDA) condividono la stessa filosofia di base (la *Separation of Concerns*), ma con una granularità differente. Possiamo tracciare un parallelo logico tra i due standard:

- **Enterprise & Information (ODP) $\approx$ CIM (MDA)**: Entrambi i livelli descrivono il business, il dominio e i dati ad alto livello, ignorando i dettagli software.
- **Computational (ODP) $\approx$ PIM (MDA)**: Entrambi descrivono l'architettura funzionale del sistema, rimanendo agnostici e indipendenti rispetto alla piattaforma tecnologica.
- **Engineering & Technology (ODP) $\approx$ PSM (MDA)**: Entrambi scendono nel dettaglio infrastrutturale, definendo i protocolli, il middleware di distribuzione e i linguaggi di programmazione specifici (es. Java, C++).

3.4 Computation Independent Model (CIM)

Il primo e più astratto livello di modellazione previsto dalla MDA è il **CIM (Computation Independent Model)**.

- Un CIM è una vista del sistema realizzata a partire dal punto di vista indipendente dal calcolo (*computation independent viewpoint*).
- A questo livello di astrazione, un CIM **non mostra alcun dettaglio** relativo alla struttura informatica o architetturale dei sistemi.

- Un CIM è talvolta definito come *modello di dominio* (*domain model*). Per la sua stesura e specifica viene rigorosamente utilizzato un *vocabolario* che risulti familiare e comprensibile ai professionisti e agli esperti di quello specifico dominio lavorativo (e non ai programmatori).
- Regola fondamentale: I requisiti espressi nel CIM devono essere *tracciabili* (*traceable*) verso i costrutti del PIM e del PSM che fisicamente li implementeranno, e viceversa.
- Facendo un parallelo con il framework visto in precedenza, un CIM è solitamente costituito da modelli che corrispondono ai punti di vista *Enterprise* e *Information* dell'ODP.

Chiarimento Accademico: Il CIM nella Pratica e la Tracciabilità

Nella realtà aziendale, il CIM è spesso rappresentato proprio dai **diagrammi BPMN** (Business Process Model and Notation) che abbiamo studiato nelle lezioni precedenti. Un manager o un esperto contabile (gli "esperti di dominio") possono leggere un BPMN senza sapere nulla di programmazione, usando il loro "vocabolario" quotidiano.

Il concetto di **Tracciabilità** (sottolineato nella slide) è cruciale: se nel CIM (diagramma BPMN) inserisco un Task chiamato "Calcola Tasse", nel PIM (diagramma UML delle classi) dovrò avere un metodo `calcolaTasse()`, e nel PSM (codice Java) dovrò avere l'effettiva implementazione `public double calcolaTasse() { ... }`. L'ingegnere deve poter tracciare una linea continua dal requisito di business iniziale fino alla singola riga di codice finale.

3.5 Platform Independent Model (PIM)

Il secondo livello di astrazione previsto dalla Model-Driven Architecture è il **PIM (Platform Independent Model)**.

- Un **PIM** è una vista di un sistema realizzata a partire dal *punto di vista indipendente dalla piattaforma* (*platform independent viewpoint*).
- Un PIM mostra un grado specifico e calcolato di indipendenza dalla piattaforma, in modo tale *da risultare idoneo per l'utilizzo con un certo numero di piattaforme differenti ma di tipo simile* (*to be suitable for use with a number of different platforms of similar type*).
- Facendo un parallelo con il framework ODP, un PIM è solitamente costituito da modelli che corrispondono alle specifiche dei punti di vista *enterprise*, *information* e, soprattutto, *computational*.

Chiarimento Accademico: Il PIM nella Pratica (UML)

Mentre il CIM (come il BPMN) è per gli analisti di business, il **PIM è il regno degli ingegneri del software**. Un classico esempio di PIM è un **Diagramma delle Classi UML** che definisce l'architettura logica di un sistema (classi, attributi,

metodi, associazioni), **senza** però specificare se sarà programmato in Java, C# o Python, e senza indicare se i dati verranno salvati su un database MySQL o MongoDB.

La frase sottolineata ("piattaforme differenti ma di tipo simile") è cruciale: un PIM orientato agli oggetti (UML) sarà facilmente trasformabile in codice Java o C# (piattaforme simili, entrambe Object-Oriented), ma sarebbe molto difficile da trasformare direttamente in un linguaggio procedurale come il C o in un linguaggio funzionale puro. Per questo l'indipendenza è "calcolata" e non assoluta.

3.6 Platform Specific Model (PSM)

Il terzo e più concreto livello di modellazione della MDA è il **PSM (Platform Specific Model)**.

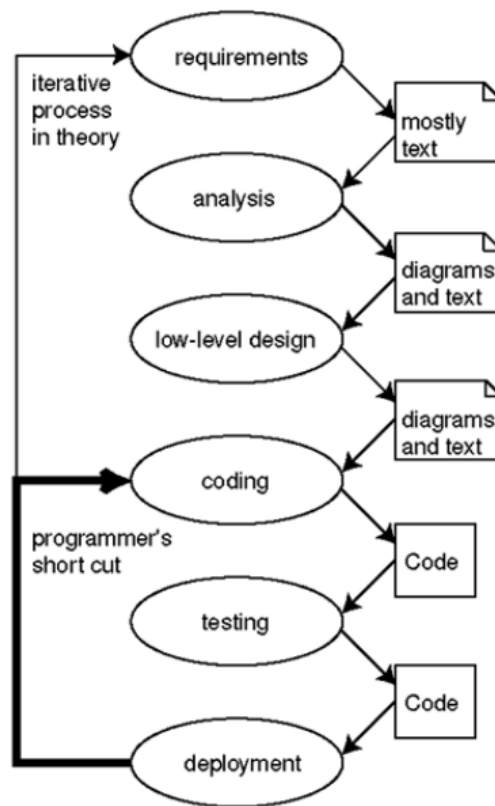
- Un **PSM** è una vista di un sistema realizzata a partire dal *punto di vista specifico della piattaforma* (*platform specific viewpoint*).
- Un PSM combina le specifiche logiche contenute nel PIM con i dettagli che specificano come quel sistema viene implementato su una particolare tipo di piattaforma.
- Un *modello di piattaforma* (*platform model*) fornisce l'insieme di concetti tecnici necessari, rappresentando i diversi tipi di componenti che costituiscono una piattaforma e i servizi da essa forniti.
 - *Esempio*: Una Classe in un modello UML che rappresenta un'entità del sistema nel PIM viene mappata (trasformata) in una **Classe Java** specifica nel PSM.

Chiarimento Accademico: Il PSM come ponte verso il Codice

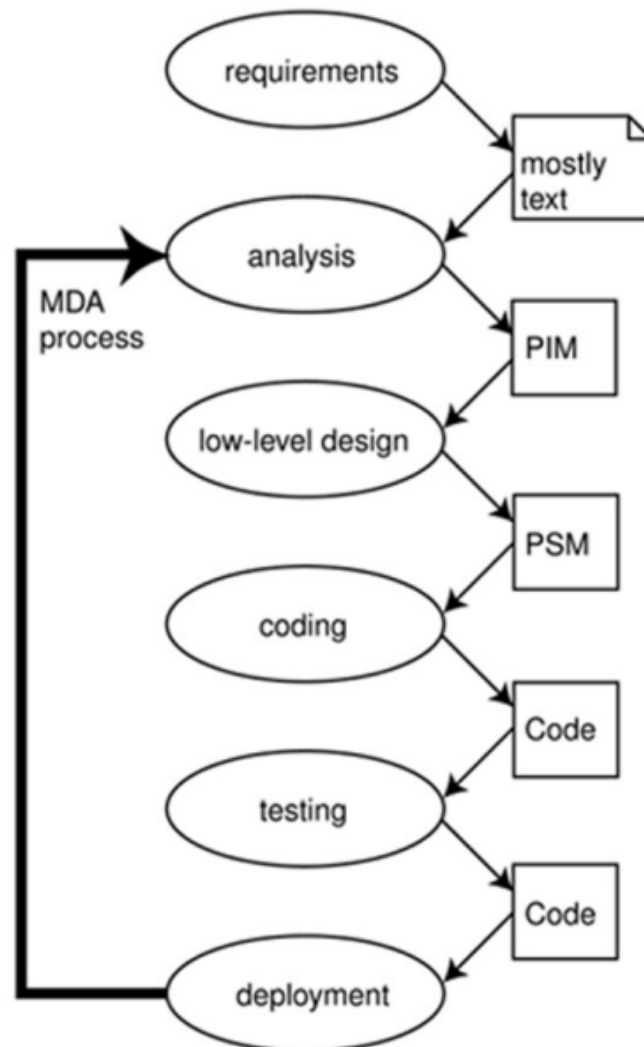
Se il PIM rappresenta il "Cosa" il sistema deve fare, il PSM rappresenta il "**Come**" lo fa su una tecnologia specifica.

Prendiamo l'esempio della Classe Java citato nella slide: nel PIM potremmo avere una classe generica **Account**. Nel momento in cui decidiamo di implementarla in Java, il PSM aggiungerà dettagli come i modificatori di accesso (**public**, **private**), i tipi di dati primitivi di Java (**int**, **double**) e le annotazioni per i framework (es. **@Entity** per JPA). In un approccio MDA maturo, il passaggio dal PSM al codice sorgente finale è spesso un'operazione di generazione automatica quasi al 100%.

Traditional sw development lifecycle



MDA development lifecycle



3.7 Mappature (Mappings)

Una mappatura MDA fornisce le *specifiche per la trasformazione* di un PIM in un PSM per una particolare piattaforma.

- Il **modello di piattaforma** è l'elemento che determina la natura stessa della mappatura.
- Il **concetto di "Mark"**: Un *mark* rappresenta un concetto specifico del PSM che viene applicato a un elemento del PIM per indicare come tale elemento debba essere trasformato.

Esempio Pratico di Mappatura: Da PIM a Java

Consideriamo l'applicazione di un mark a una classe logica per guidarne la generazione del codice:

- Lo stereotipo «**Entity**» è un mark che può essere applicato alle classi in un PIM.
- Una classe **Person** contrassegnata come «**Entity**» nel PIM potrebbe essere mappata a una **Classe Java** nel PSM secondo le seguenti regole di trasformazione:
 - Gli attributi (es. *name* e *birthdate*) vengono trasformati in **campi privati** (*private fields*).
 - Vengono generati automaticamente un **costruttore** e i corrispondenti metodi **getter e setter** per garantire l'incapsulamento.
 - Gli attributi vengono mappati su tipi Java specifici: ad esempio, *name* diventa una **String** e *birthdate* un oggetto **LocalDate**, riflettendo una scelta tecnologica specifica per la gestione delle date sulla piattaforma Java.

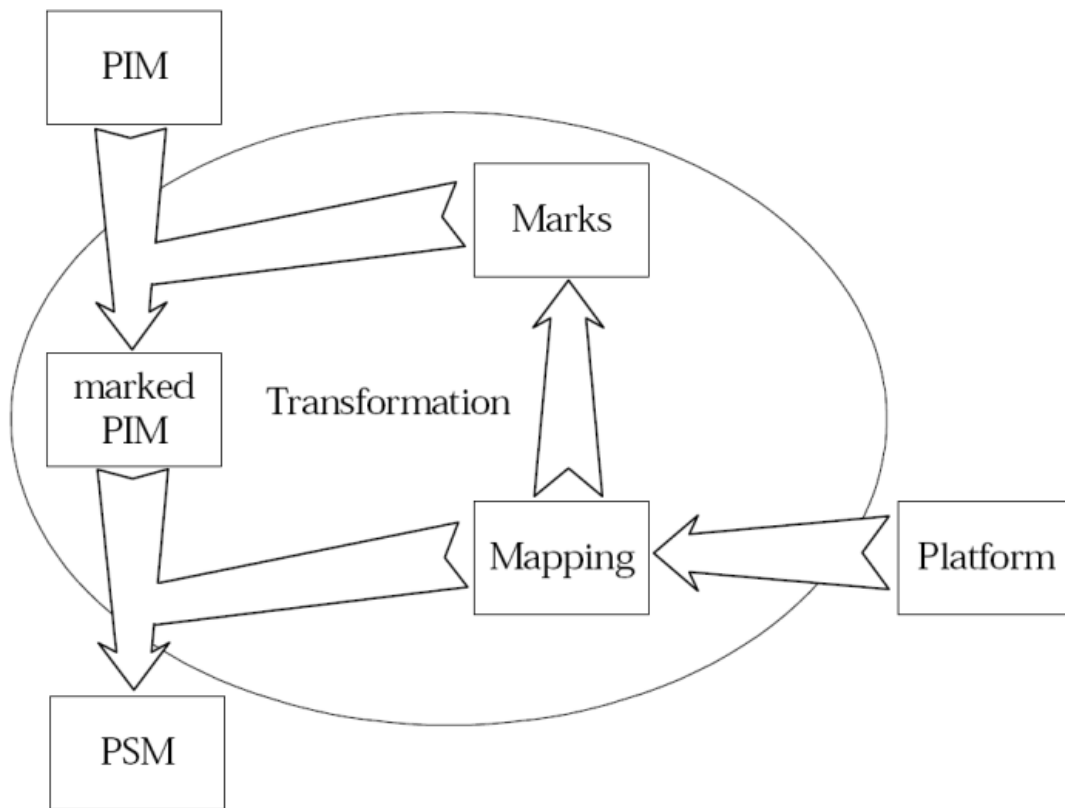
Chiarimento Accademico: Il Ruolo dei Mark e del Profilo UML

Il concetto di "Mark" citato nella slide è tecnicamente implementato in UML attraverso gli **Stereotipi** e i **Profili**.

Nel tuo progetto sul satellite, quando applichi lo stereotipo «*software*» a una classe, stai applicando un **Mark**. Questo segnale dice al tuo motore di trasformazione (Acceleo): "Questa classe non è un pezzo di ferro (hardware), ma deve diventare un file Java". La mappatura definisce poi che ogni attributo UML di quella classe debba diventare un campo privato con relativo getter/setter nel codice generato, esattamente come descritto nell'esempio della classe **Person**.

Model Transformation

(*marking approach*)



3.8 Principi e Standard della MDA di OMG

La **Model-Driven Engineering (MDE)** è un approccio metodologico che affronta la complessità intrinseca dello sviluppo di sistemi focalizzandosi sull'utilizzo di **modelli formali**.

- I modelli astratti, specificati all'inizio del ciclo di vita del sistema, vengono forniti in input a delle **model transformations** (trasformazioni di modello).
- Tali trasformazioni generano modelli a livelli di astrazione via via più bassi, fino a ottenere modelli rifiniti in modo incrementale che possono essere resi **eseguibili**.
- La **MDA** dell'OMG (rilasciata nel 2003) rappresenta una delle iniziative più rilevanti guidate dal paradigma MDE e fornisce i seguenti standard fondamentali:
 - **Meta Object Facility (MOF)**: Utilizzato per specificare metamodelli tecnologicamente neutri (ovvero modelli usati per descrivere altri modelli).
 - **XML Metadata Interchange (XMI)**: Fornisce un insieme di regole per la serializzazione di metamodelli basati su MOF.

- **Query/View/Transformation (QVT):** Standard per la specifica di trasformazioni **modello-a-modello** (*model-to-model*).
- **MOF Model To Text (MOFM2T):** Standard per la specifica di trasformazioni **modello-a-testo** (*model-to-text*), come la generazione di codice sorgente.

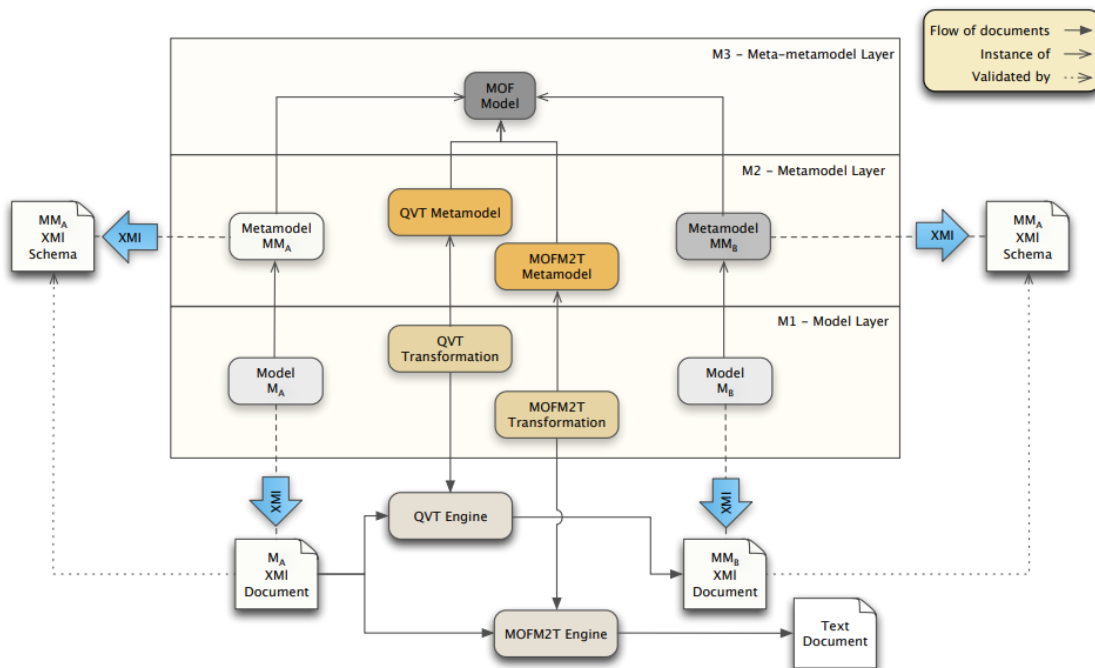
Chiarimento Accademico: L'Architettura a 4 Livelli del MOF

Per comprendere il MOF, l'ingegnere deve pensare a una gerarchia di astrazione:

1. **M0 (Oggetti reali):** Il sistema fisico in funzione.
2. **M1 (Modello):** Il diagramma UML (es. il tuo progetto sul satellite). Conforme a M2.
3. **M2 (Metamodello):** Lo standard UML stesso (definisce cosa sia una "Classe" o un "Attributo"). Conforme a M3.
4. **M3 (Meta-Metamodello):** Il MOF. È il linguaggio "radice" usato per definire l'UML e altri linguaggi. Il MOF è auto-descrittivo: è definito usando se stesso.

Nel tuo progetto, utilizzerai **Acceleo**, che è un'implementazione dello standard **MOFM2T**, per trasformare il tuo modello (M1) direttamente in testo (codice Java).

OMG's MDA principles and standard



3.9 XMI in Eclipse

L'ambiente di sviluppo Eclipse implementa gli standard della Model-Driven Architecture attraverso l'utilizzo di framework specifici:

- **Eclipse Modeling Framework (EMF):** Eclipse utilizza l'EMF come propria implementazione della **Meta Object Facility (MOF)**.
- **ECORE:** Rappresenta il metamodello di riferimento dell'EMF ed è concettualmente simile allo standard **EMOF** (*Essential MOF*) dell'OMG.
- Il progetto EMF mette a disposizione strumenti avanzati per:
 - Creare metamodelli **ECORE**.
 - Generare schemi **XMI** per i modelli basati su Ecore.
 - Creare modelli conformi ai metamodelli definiti.
 - Leggere e scrivere tali modelli utilizzando il formato di serializzazione **XMI**.
- Nota tecnica: È importante sottolineare che **Ecore XMI non è identico all'OMG XMI**. Esistono alcune piccole differenze strutturali dovute alle discrepanze intrinseche tra le specifiche di Ecore e quelle di EMOF.

Chiarimento Accademico: Ecore vs. MOF

In ambito accademico e professionale, è fondamentale distinguere tra lo standard e la sua implementazione. Mentre l'OMG definisce il **MOF** come standard teorico universale per il metamodello, l'ecosistema Eclipse ha creato **Ecore** come versione pragmatica e ottimizzata del MOF.

Nel tuo progetto, quando utilizzerai strumenti come Papyrus per creare il modello UML del satellite, starai lavorando sotto il cofano con modelli basati su Ecore. Anche se utilizzerai file con estensione `.uml`, la loro struttura interna di serializzazione seguirà le regole XMI di Eclipse per permettere a strumenti come **Acceleo** di leggere il modello e generare correttamente il codice Java richiesto.

Ecore vs eMOF

<u>Ecore</u>	<u>EMOF</u>
http://www.eclipse.org/emf/2002/Ecore	http://schema.omg.org/spec/mof/2.0/emof.xml
<u>EObject</u>	Object
<u>EModelElement</u>	Object
<u>eAnnotations</u> map reference	<u>xmi:Extension</u>
<u>ENamedElement</u>	<u>NamedElement</u>
name attribute	name property
<u>EPackage</u>	Package
<u>nsURI</u> attribute	<u>uri</u> property
<u>eSubpackages</u> reference	<u>nestedPackage</u> property
<u>eSuperPackage</u> reference	<u>nestingPackage</u> property
<u>eClassifiers</u> reference	<u>ownedType</u> property
<u>nsPrefix</u> attribute	<u>xmi:Extension</u>
<u>EStructuralFeature</u>	Property
changeable attribute	<u>isReadOnly</u> property (boolean not)
defaultValueLiteral attribute	default property
derived attribute	<u>isDerived</u> property
<u>eContainingClass</u> reference	class property
<u>EAttribute</u>	Property
iD attribute	<u>isID</u> property
<u>EReference</u>	Property
containment attribute	<u>isComposite</u> property
<u>eOpposite</u> reference	opposite property
<u>resolveProxies</u> attribute	<u>xmi:Extension</u>

4 Capitolo 4: Model-to-Text Transformation

4.1 Introduzione alle Trasformazioni Model-to-Text (M2T)

Le trasformazioni **Model-to-Text** vengono introdotte nel processo di sviluppo per automatizzare la generazione di diversi tipi di artefatti testuali a partire dai modelli. Gli output più comuni di questo processo includono:

- **Codice sorgente** dei programmi (es. classi Java, C++, Python).
- **Documentazione** tecnica di progetto.
- **Casi di test** (*test cases*) per la validazione del sistema.
- **Serializzazione del modello** in formati come XML.

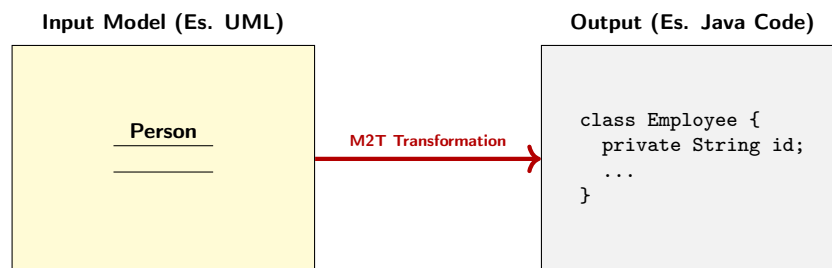


Figura 8: Esempio concettuale di trasformazione da modello UML a codice Java

4.2 Lo standard MOF M2T ed Acceleo

Il **MOF Model to Text (mof2text)** è lo standard ufficiale della MDA per la specifica delle trasformazioni da modello a testo.

- Lo standard definisce come tradurre un modello in vari artefatti testuali come codice, specifiche di deployment, report o documenti.
- Essenzialmente, il MOF M2T affronta il problema di come trasformare un modello (una struttura a grafo) in una *rappresentazione testuale linearizzata*.

Acceleo: L'implementazione dello standard

Acceleo è una delle implementazioni più diffuse della specifica MOFM2T definita dall'OMG.

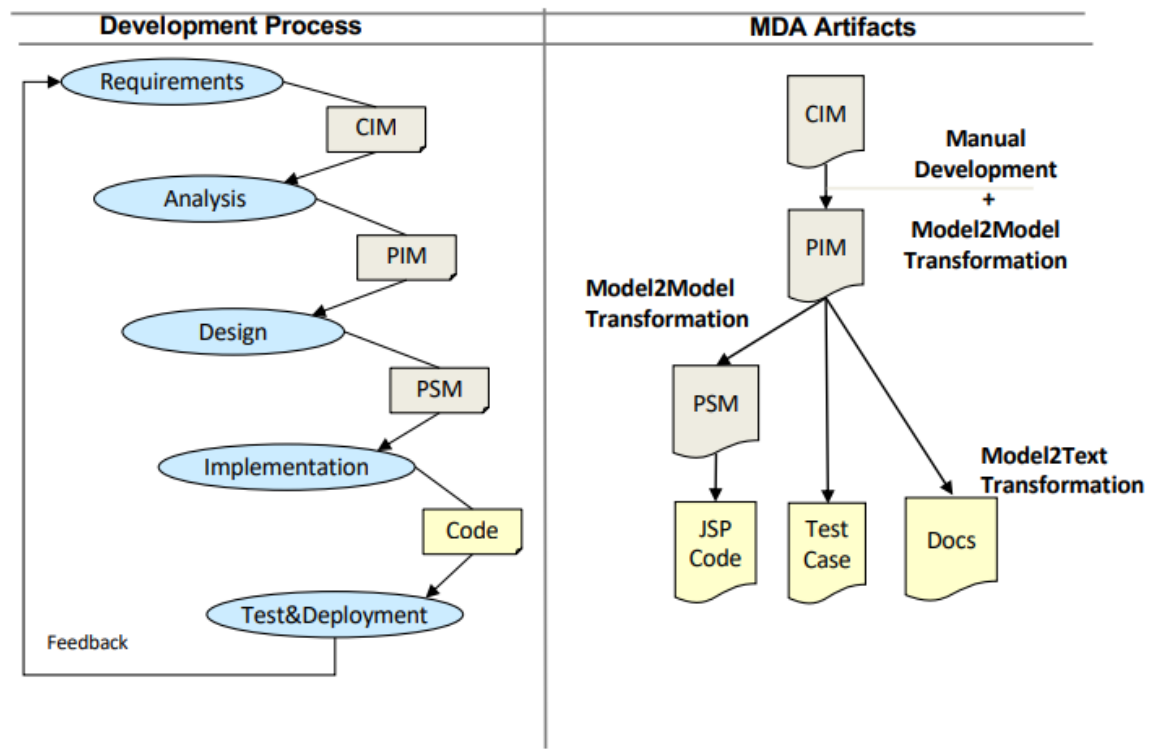
- Il linguaggio utilizzato da Acceleo è denominato **MTL** (*Model Transformation Language*).
- All'interno di un modulo Acceleo, la logica di trasformazione è composta principalmente da due tipi di strutture: **template** e **query**.
- Acceleo permette di utilizzare espressioni basate su un sottoinsieme del linguaggio **OCL** (*Object Constraint Language*) per interrogare ed esplorare i modelli di input.

Perché MTL è basato su Template?

A differenza delle trasformazioni Modello-a-Modello (M2M) che sono spesso basate su regole (*rules*), Acceleo lavora per **Template**. Immagina di avere una classe "scheletro" Java vuota: il template definisce la struttura fissa (parole chiave come `public class`), mentre le parti variabili (come il nome della classe o gli attributi) vengono "iniettate" dinamicamente dal motore Acceleo estraendole dal modello UML tramite query OCL.

Questo approccio rende la generazione del codice molto simile al concetto di "stampa unita" o ai template engine web, rendendo il codice generato pulito e facile da formattare.

Model-to-text transformations



4.3 Approcci alla Generazione del Codice

Per la generazione di codice (o più in generale di testo) a partire dai modelli, possono essere utilizzati due approcci distinti:

- **Utilizzo di linguaggi general-purpose:** L'impiego di linguaggi come Java o C++ per manipolare direttamente la serializzazione basata su XML del modello di input.

- **Utilizzo di linguaggi di trasformazione Model-to-Text (M2T):** L'impiego di linguaggi specifici basati sullo standard **MOFM2T** della Model-Driven Architecture (MDA).

Linguaggi General Purpose (Approccio Programmatico)

Una trasformazione M2T basata su un linguaggio di programmazione generico segue solitamente un processo strutturato in tre fasi:

1. **Caricamento:** Il file XMI viene caricato in memoria.
2. **Elaborazione:** I modelli vengono processati per produrre il codice attraverso il parsing della struttura del modello, l'estrazione delle informazioni e il salvataggio del codice risultante in variabili di tipo stringa.
3. **Scrittura:** La variabile stringa viene persistita su un file fisico tramite flussi di dati (*streams*).

Pro (Vantaggi)	Contro (Svantaggi)
Non richiede la conoscenza di framework specifici, strumenti complessi o nuovi linguaggi di modellazione.	[leftmargin=*, nosep] • Impossibilità di separare la produzione di codice statico da quello dinamico. • La struttura dell'output non si basa su un template gestibile. • Mancanza di un linguaggio di interrogazione dichiarativo. • Assenza di funzionalità di base riutilizzabili.

Tabella 1: Analisi dell'approccio basato su linguaggi general-purpose

4.4 Approfondimento sullo standard MOF M2T

Come anticipato, il **MOF Model to Text** è lo standard definito dall'OMG per colmare le lacune degli approcci programmatici tradizionali.

- **Linearizzazione:** Lo standard affronta la sfida tecnica di trasformare un modello, che per sua natura è una struttura a grafo non lineare, in una ***rappresentazione testuale linearizzata*** pronta per la compilazione o la lettura.
- **Sintassi di Acceleio:** In quanto implementazione del MOFM2T, Acceleio utilizza il linguaggio **MTL** (*Model Transformation Language*).
- **Modularità:** La logica di trasformazione è incapsulata in **moduli** che contengono:
 - **Templates:** Per definire la struttura dello scheletro del file di output.
 - **Queries:** Per estrarre dati complessi dal modello utilizzando espressioni **OCL**.

L'importanza dei Template in Acceleo

Il vantaggio competitivo di Acceleo risiede nella gestione dei template. In un approccio programmatico (Java), il programmatore deve concatenare manualmente decine di stringhe, rendendo il codice difficile da mantenere. In Acceleo, il template permette di visualizzare il file finale così come apparirà, inserendo solo dei "segnaposto" dinamici. Questo garantisce che la struttura dell'output sia sempre coerente con il metamodello di destinazione.

4.5 L'Approccio basato su Template (M2T)

A differenza dell'approccio programmatico, un approccio basato su **M2T** utilizza il concetto di **template** per gestire la generazione. Un template è composto da frammenti di testo statico e da *meta-marker* in grado di interrogare fonti di dati aggiuntivi.

Le caratteristiche principali di questo approccio sono:

- **Frammenti di testo:** Vengono inclusi *verbatim* (testualmente) nel file di output.
- **Marker:** Devono essere interpretati e valutati dal motore di trasformazione.
- **Interrogazione del modello:** Può avvenire in modo **dichiarativo** (usando linguaggi come OCL, XPath o SQL) o **imperativo** (usando Java o C#).
- **Template Engine:** Si occupa di sostituire a runtime i meta-marker con i dati reali estratti dal modello sorgente per produrre i file di output.

4.6 UML e le sue limitazioni

Nonostante l'UML sia lo standard de facto per la modellazione, presenta delle limitazioni intrinseche quando si tratta di definire la logica di business in modo preciso:

- L'UML è eccellente per modellare la **struttura** (Diagrammi delle Classi, attributi, associazioni) e il **comportamento** (Macchine a Stati, Diagrammi di Sequenza).
- Tuttavia, i diagrammi UML sono prevalentemente **grafici** e non possono esprimere con precisione alcune **regole di business**.
 - *Esempio:* "Il saldo di un conto non deve mai essere negativo".
 - *Esempio:* "Uno studente deve essere iscritto ad almeno un corso".

Per superare questi limiti, è necessario un **linguaggio testuale preciso** che integri l'UML.

4.7 Object Constraint Language (OCL)

L'**OCL** è un linguaggio formale e dichiarativo utilizzato insieme all'UML per esprimere vincoli precisi sui modelli.

- **Cosa permette di definire:**

- **Invarianti:** Condizioni che devono essere **sempre vere** per l'elemento del modello.
- **Precondizioni:** Condizioni che devono essere soddisfatte **prima** dell'esecuzione di un'operazione.
- **Postcondizioni:** Condizioni che devono risultare vere **dopo** l'esecuzione di un'operazione.

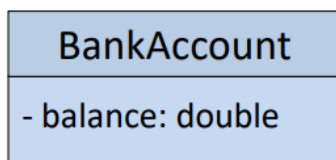
- **Proprietà chiave:**

- **Dichiarativo:** Descrive *cosa* deve valere, non come calcolarlo.
- **Side-effect free:** La valutazione di un'espressione OCL **non modifica** mai il modello.
- **Univoco e formale:** Elimina le ambiguità tipiche del linguaggio naturale.

OCL nel MBSE

Nell'ambito della *Model-Based Systems Engineering* (MBSE), l'OCL è uno strumento vitale perché migliora drasticamente la **precisione**, la **coerenza** e permette la **validazione precoce** dei modelli, riducendo gli errori nelle fasi avanzate dello sviluppo.

Simple OCL Example



Constraint in natural language:

"The balance of a bank account must never be negative"

Constraint in OCL:

context BankAccount
inv NonNegativeBalance:
self.balance >= 0

context: the constraint applies to all *BankAccount* instances
inv: invariant (always true)
self: the current object
dot notation: used to access class attributes

4.8 Vantaggi dell'approccio M2T

L'utilizzo di un linguaggio di trasformazione dedicato e basato sullo standard **MOFM2T** offre vantaggi significativi rispetto all'elaborazione manuale tramite linguaggi general-purpose:

- **Netta separazione tra codice statico e dinamico:**
 - I template permettono di distinguere chiaramente il testo normale (statico) dal codice dinamico, il quale viene descritto in modo puntuale tramite i *meta-marker*.
- **Struttura dell'output esplicita:**
 - La struttura primaria del template corrisponde visivamente alla struttura finale del file di output.
 - La logica di computazione necessaria per l'estrazione dei dati è integrata (*embedded*) direttamente all'interno di questa struttura.
- **Linguaggio di interrogazione dichiarativo:**
 - Viene impiegato l'**OCL** (*Object Constraint Language*) per interrogare i modelli di input in modo formale e non ambiguo.
- **Funzionalità di base riutilizzabili:**
 - Il framework offre supporto nativo per operazioni comuni, come la lettura dei modelli e la serializzazione automatica del testo su file fisici.

Acceleo: Stato dell'Arte nelle Trasformazioni M2T

Acceleo rappresenta oggi una delle implementazioni più rilevanti e mature dello standard **MOFM2T** dell'OMG. Nel contesto del tuo progetto, Acceleo ti permetterà di definire dei template MTL che "scansionano" il diagramma delle classi UML del satellite. Per ogni classe marcata con lo stereotipo «*software*», il motore Acceleo utilizzerà query OCL per estrarre nomi, attributi e note, inserendoli nello scheletro di un file `.java` pre-formatto. Questo garantisce che il codice prodotto sia modulare, privo di errori di sintassi ripetitivi e perfettamente allineato al modello originale.

4.9 Struttura di un Generatore Acceleo

Un generatore Acceleo è un'entità modulare composta da diversi file denominati **moduli**.

- **Il file `.mtl`:** Ogni modulo è salvato con estensione `.mtl` e include i seguenti elementi strutturali fondamentali:
 - **Template:** Il blocco principale utilizzato per definire la struttura del codice da generare.

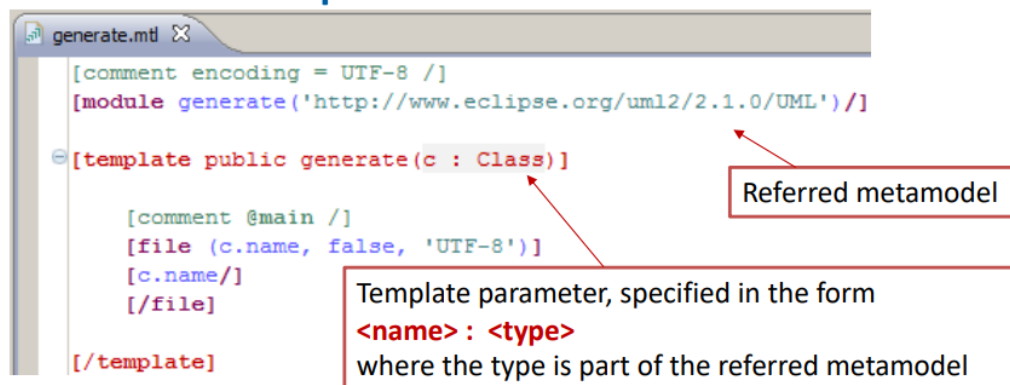
- **Query:** Un'espressione **OCL** dedicata al recupero delle informazioni necessarie dal modello sorgente (input).
- **Dichiarazione del Modulo:** Ogni modulo è identificato dalla keyword **module** seguita dal nome e dagli URI dei metamodeli di riferimento:

Esempio di sintassi MTL

```
[module <module_name>('metamodel_URI_1', 'metamodel_URI_2')]
...
[/module]
```

- **Ispezione a Runtime:** I metamodeli riferiti nella dichiarazione forniscono al motore di trasformazione il meccanismo necessario per ispezionare le istanze dei modelli reali durante l'esecuzione.
- **Riutilizzo del codice:** I moduli Acceleo sono progettati per essere altamente riutilizzabili:
 - Un template può **includere** (*include*) un template esistente.
 - Un template può **sovrascrivere** (*override*) o **estendere** (*extend*) un template esistente, permettendo la personalizzazione delle trasformazioni senza modificare il codice originale.

Template definition



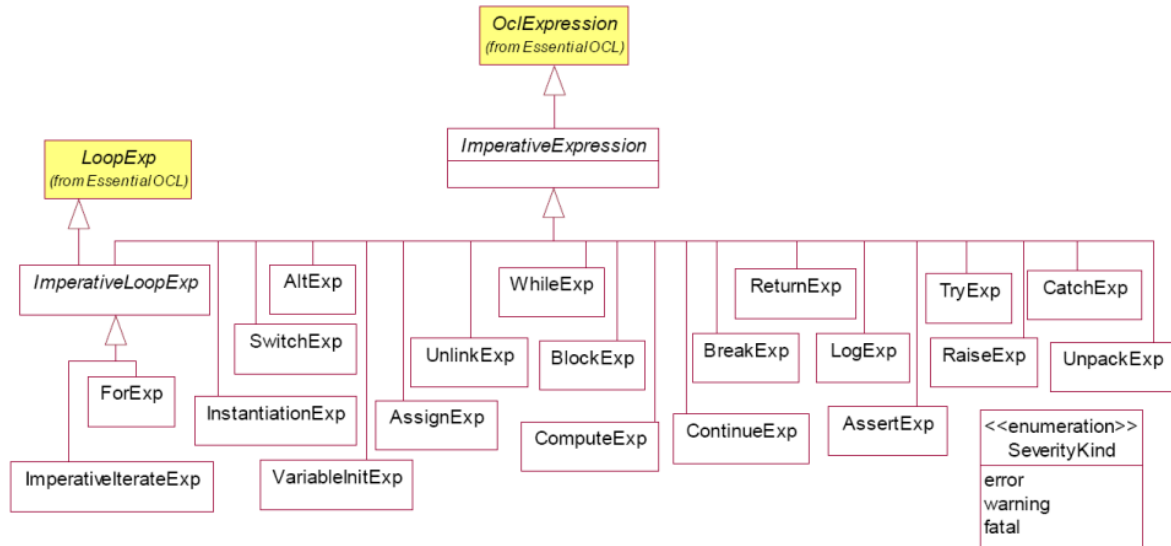
Templates are sets of Acceleo statements used to generate text. They are delimited by the **[template...]** **[/template]** tags.

Templates can also have **optional elements**:

- Overriding: *overrides*
- Pre-conditions (or guard conditions): *?(<guard>)*
- Post-treatments: *post(<condition>)*
- Variable initializations: *{ <varName>:<type> = <val>; }*

Imperative OCL Package

<https://www.omg.org/spec/OCL/2.0/PDF>



4.10 Collezioni OCL (OCL Collections)

Il linguaggio OCL gestisce i gruppi di elementi attraverso diverse tipologie di collezioni, ognuna con caratteristiche matematiche e logiche specifiche per la navigazione dei modelli:

- **Set (Insieme):** Rappresenta il concetto matematico di insieme.
 - Non permette elementi duplicati.
 - Non garantisce un ordine specifico tra gli elementi.
 - È un'istanza del metatipo **SetType**.
- **OrderedSet (Insieme Ordinato):** Un insieme in cui la posizione degli elementi è rilevante.
 - Non permette elementi duplicati.
 - Gli elementi mantengono un ordine sequenziale.
 - È un'istanza del metatipo **OrderedSetType**.
- **Bag (Multinsieme):** Una collezione che ammette ripetizioni.
 - Gli elementi duplicati sono ammessi (un oggetto può comparire più volte).
 - Non è definito alcun ordine tra gli elementi.
 - È un'istanza del metatipo **BagType**.
- **Sequence (Sequenza):** Una collezione ordinata che ammette ripetizioni.
 - Gli elementi sono ordinati.

- Un elemento può far parte della sequenza più di una volta.
- È un'istanza del metatipo `SequenceType`.

Tabella Comparativa delle Collezioni

Per scegliere la collezione corretta durante la scrittura di una query Acceleo, è utile consultare questo schema:

Tipo di Collezione	Ordinata?	Duplicati Ammessi?
Set	No	No
OrderedSet	Sì	No
Bag	No	Sì
Sequence	Sì	Sì

Ad esempio, se in un modello UML interroghiamo gli attributi di una Classe, OCL restituirà solitamente una **OrderedSet**, poiché in UML l'ordine degli attributi è preservato ma non possono esistere due attributi identici nella stessa classe.

Template definition: examples

```
[comment Generates the java code for a class property that belongs to an association and is ordered /]
[template public genAssociation(p : Property) ? (owningAssociation <> null and isOrdered)]
[/template]
```

```
generate.mtl
[comment encoding = UTF-8 /]
[module generate('http://www.eclipse.org/uml2/2.1.0/UML')]
[template public generate(c : Class) { javaName : String = javaName(); }]

[comment @main /]
[file (c.fullFilePath(), false, 'UTF-8')]
package [packageName()];

import java.util.List;

public class [javaName/] {

    [for (att : Property | ownedAttribute)]
    private [javaType()] [javaName/];

    public [javaType()] get[javaName.toUpperFirst()]/() {
        return [javaName/];
    }

    public void set[javaName.toUpperFirst()]/([javaType()] [javaName/]) {
        this.[javaName/] = [javaName/];
    }

[/for]
```

```
[template public javaName (c : Class) post (trim()) ]
[name.toUpperFirst()/]
[/template]
```

✓ dot notation: <type>.<operation>
invokes the operation (OCL methods, mappings, queries...) over the element instance

4.11 Query e Servizi in Acceleo

Le **query** rappresentano lo strumento principale per estrarre informazioni dal modello sorgente durante la generazione. Esse restituiscono valori singoli o collezioni di valori e utilizzano il linguaggio **OCL** racchiuso in appositi tag.

76

- **Sintassi delle Query:** Una query è definita dalla keyword `[query ... /]` e deve essere progettata per restituire sempre lo stesso valore se invocata con i medesimi argomenti.

Sintassi Formale Query

```
[query public <query_name>( [<param> : <type>] ) : <ret_type> =
<expression> /]
```

- **Notazione a Freccia (->):** In OCL, la notazione `<collezione> -> <operazione>` indica che l'operazione a destra viene applicata a ogni singola istanza della collezione a sinistra.
- **Esempio Pratico:** Per estrarre gli attributi pubblici di una classe, si utilizza la query:
`c.attribute->select(visibility = VisibilityKind::public).`

Invocazione di Servizi Esterni

In alcuni scenari, una trasformazione può richiedere operazioni logiche o matematiche troppo complesse per essere implementate esclusivamente con le direttive standard di Acceleo.

- Acceleo permette l'**invocazione di servizi esterni** scritti in linguaggio **Java**.
- Questi servizi vengono incapsulati all'interno di query MTL (wrapping).
- L'esecuzione di un servizio corrisponde all'invocazione di un **metodo pubblico di una classe Java**, la quale deve essere preventivamente importata nel generatore.

Sintassi Invocazione Servizio

```
[query public <query_name>( [<params>] ) : <ret_type> =
invoke('<Classe_Java>', '<segnatura_metodo>', {<lista_parametri>})
/]
```

Perché usare i Servizi Java?

Sebbene OCL sia molto potente per navigare i modelli, non è un linguaggio di programmazione computazionale. Se la tua trasformazione per il progetto del satellite richiedesse, ad esempio, di calcolare la memoria totale occupata basandosi su una formula fisica o di formattare una stringa secondo uno standard proprietario complesso, risulterebbe molto più semplice scrivere quella logica in una classe Java ausiliaria e "agganciarla" al template Acceleo tramite un servizio. In questo modo si mantiene il template MTL pulito, delegando la computazione pesante al linguaggio Java.

4.12 Elementi Strutturali del Linguaggio (MTL)

Oltre ai template e alle query, il linguaggio MTL di Acceleo fornisce costrutti specifici per la gestione dei file, l'iterazione e la logica condizionale.

Generazione di File

- **Tag** `[file] ... [/file]`: Sono utilizzati per indirizzare e generare il risultato di una trasformazione M2T all'interno di un file fisico reale.

Sintassi Blocco File

```
[file(<uri_expression>, <append_mode>, <output_encoding>)]  
...  
[/file]
```

- `<uri_expression>`: Definisce il nome e il percorso del file di output.
- `<append_mode>`: Un parametro opzionale che indica se il testo generato deve essere accodato al file esistente (`true -> append`) o se deve sostituirne il contenuto (`false -> replace`).
- `<output_encoding>`: Un parametro opzionale che indica l'encoding da utilizzare per il file generato (es. `'UTF-8'`).

Iterazioni e Condizioni

- **Tag** `[for] ... [/for]`: Sono impiegati per creare cicli iterativi (*for loops*) sulle collezioni OCL.
 - *Sintassi leggera (Light syntax)*: `[for (<iterableExpr>)] ... [/for]`.
 - *Sintassi completa (Full syntax)*: `[for (<iterator> : <iteratorType> | <iterableExpr>)] ... [/for]`.
- **Tag** `[if]...[else]...[/if]`: Permettono di specificare l'esecuzione condizionale di porzioni di template in base a un'espressione logica.
 - *Sintassi*: `[if (<ocl expression>)] ... [else] ... [/if]`.

Variabili, Commenti e Aree Protette

- **Tag** `[let ...] ... [/let]`: Un blocco `let` viene utilizzato per creare una variabile e assegnarle un valore temporaneo. In Acceleo le variabili sono rigorosamente *final*: il loro valore non può più essere modificato dopo la prima assegnazione.
 - *Sintassi*: `[let <varName>:<VarType>=<ocl expression> ] ... [/let]`.
- **Tag** `[comment /]`: Definisce un blocco di commenti invisibile nel codice finale generato.
 - *Sintassi*: `[comment .... /]`.

- **Tag `[protected]...[/protected]`:** Identifica un'area protetta (*protectedArea-Block*), ovvero una porzione di testo che necessita di essere preservata attraverso molteplici esecuzioni di trasformazione model-to-text.
 - *Sintassi:* `[protected (<ocl expression>)] block_expression [/protected]`.

L'importanza cruciale del tag `[protected]`

In un ciclo di sviluppo Model-Driven, i modelli evolvono e il codice viene generato ripetutamente. Immagina di generare una classe Java tramite Acceleo e poi di scrivere manualmente l'implementazione di un algoritmo all'interno di un suo metodo. Senza il tag `[protected]`, alla successiva rigenerazione del modello (ad esempio perché hai aggiunto un nuovo attributo alla classe UML), Acceleo sovrascriverebbe l'intero file, distruggendo il tuo lavoro manuale!

Includendo il blocco `[protected ('id_univoco')]` all'interno del template, Acceleo inserirà dei commenti speciali nel file Java. Da quel momento, il motore saprà che qualsiasi testo inserito manualmente all'interno di quegli indicatori deve sopravvivere (*preserved*) a tutte le generazioni successive.

5 Capitolo 5: Acceleio in Practice

5.1 Panoramica delle Attività (Outline)

Questa sezione del corso si concentra sull'applicazione pratica dei concetti di Model-Driven Engineering. Le attività principali prevedono:

- **Configurazione dell'ambiente di sviluppo:**
 - Download e installazione di Eclipse (versione con i Modeling Tools integrati).
 - Installazione dei plugin necessari: Papyrus, QVTo e Acceleio.
- **Creazione e applicazione di un Profilo.**
- **Esempio di trasformazione M2T (Model-to-Text) con Acceleio.**

5.2 L'Evoluzione dell'UML: I Profili (UML Profile)

Un linguaggio di uso generale (*general-purpose*) come l'UML standard potrebbe non essere sufficientemente espressivo per descrivere in modo accurato le applicazioni appartenenti a uno specifico dominio.

Di fronte a questa limitazione, l'ingegnere del software ha due possibili alternative:

1. **Adozione di un nuovo linguaggio:** Creare un linguaggio da zero. Tuttavia, un nuovo linguaggio di modellazione non verrebbe supportato dagli strumenti commerciali UML preesistenti.
2. **Estensione dell'UML esistente:** Estendere la sintassi e la semantica dell'UML standard. Questo approccio è la via preferenziale e si realizza tramite l'adozione degli ***UML Profile*** (Profili UML).

Struttura e Componenti di un Profilo UML

Il pacchetto del Profilo UML fornisce un insieme di artefatti UML progettati per specificare un modello MOF (in altre parole, funge da ***metamodello***). Questi artefatti permettono di gestire i concetti, le notazioni e le problematiche tipiche di un **particolare dominio applicativo e/o di una specifica piattaforma di implementazione**.

Tecnicamente, i profili UML si basano su tre meccanismi di estensione fondamentali:

- **Stereotypes (Stereotipi):** Estendono le metaclassi UML (es. *Class*, *Component*) assegnando loro un nuovo significato semantico.
- **Constraints (Vincoli):** Regole aggiuntive (spesso scritte in OCL) che restringono l'uso degli elementi per garantirne la validità nel nuovo dominio.
- **Tagged values (Valori etichettati):** Attributi aggiuntivi associati agli stereotipi per memorizzare metadati specifici.

Collegamento Pratico: Il Profilo per il tuo Progetto

Questa teoria è la base esatta della ****Fase 1 e 2 del tuo progetto****. Poiché l'UML di base ha solo il concetto generico di "Classe", tu creerai un Profilo UML per estendere questo concetto. Creerai due **Stereotipi** («hardware» e «software») per specificare il dominio (il sistema satellitare). Inoltre, aggiungerai un **Tagged Value** chiamato **note** a questi stereotipi, in modo da poter inserire dei commenti testuali direttamente dentro i blocchi UML. Senza i Profili UML, saresti costretto a inventare un linguaggio grafico da zero!

5.3 Trasformazioni e Profili UML

Una trasformazione di modello prende in input un modello sorgente (*source model*) e produce come risultato:

- Un modello di destinazione richiesto (*model-to-model*).
- Un documento di testo o codice sorgente richiesto (*model-to-text*).

Il limite dell'UML nativo

Nella pratica ingegneristica, le informazioni fornite *nativamente* da un modello sorgente UML standard spesso **non sono sufficienti** per guidare completamente e correttamente una trasformazione.

- Per superare questo ostacolo, l'utilizzo di un **Profilo UML** si rivela uno strumento estremamente efficace per **arricchire** il modello sorgente con i metadati necessari.

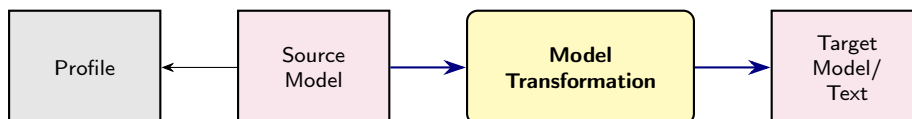
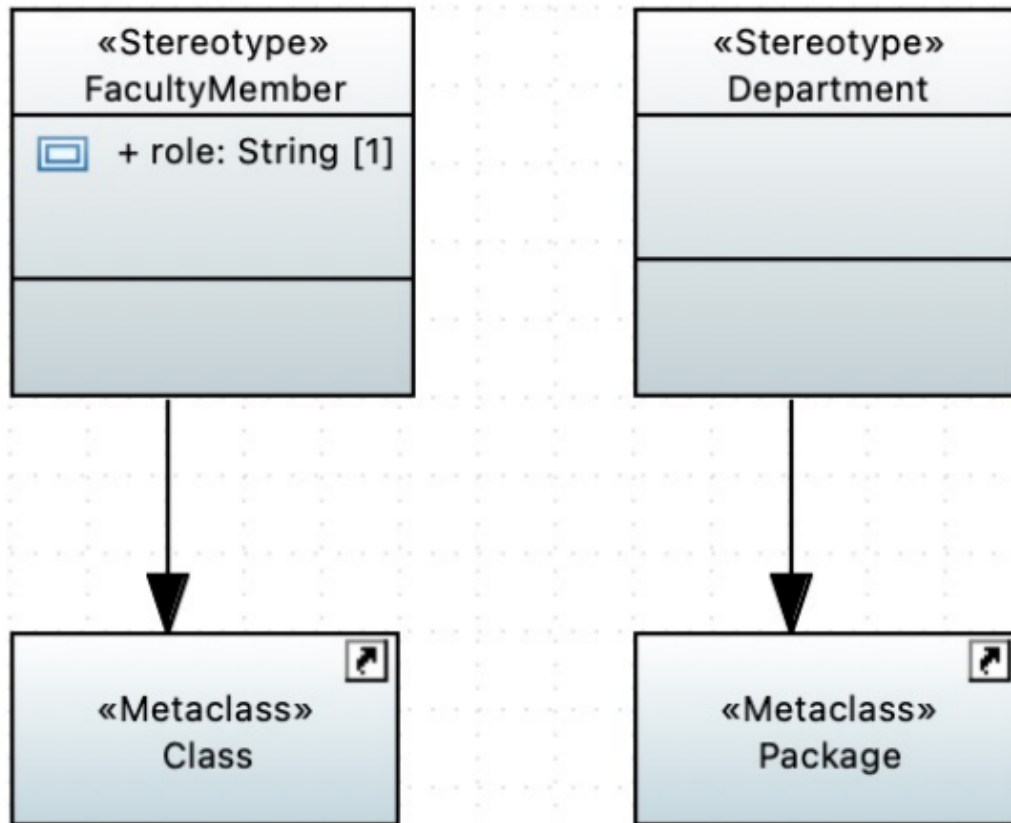


Figura 9: Il ruolo del Profilo UML nel processo di Trasformazione

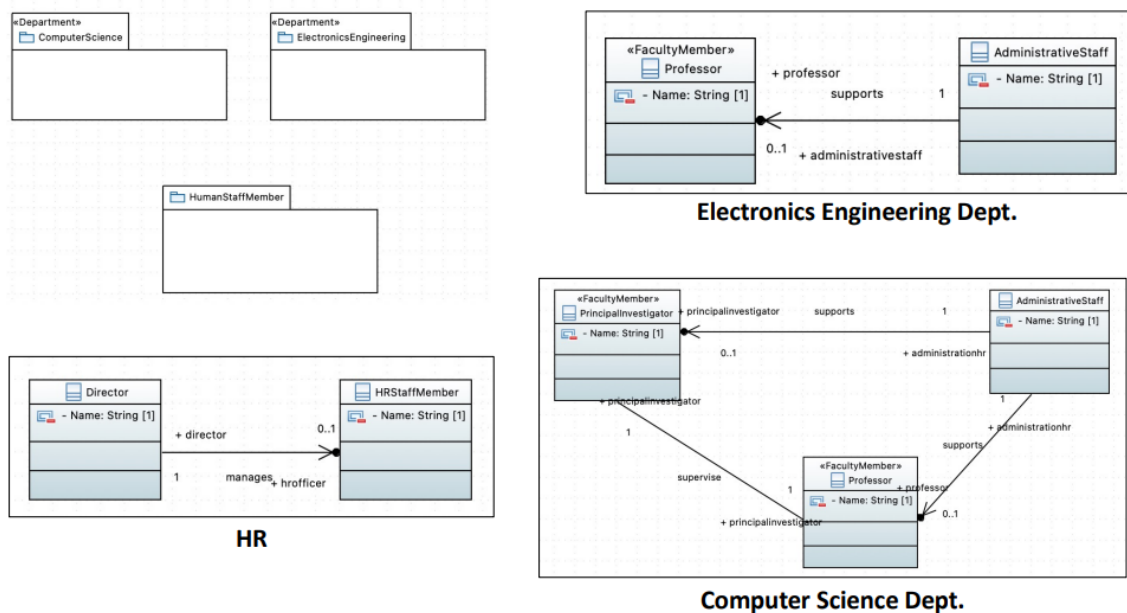
Analisi applicata al Task 1: Perché arricchire il modello?

Questa slide spiega l'esatta logica della ****Fase 4**** del tuo progetto. Il requisito ti chiede di generare l'implementazione Java "limitatamente ai soli componenti software". Se tu usassi solo l'UML nativo, avresti semplicemente una serie di "Classi" (ComSys, AC, EC, Sensors, RW). Il motore di trasformazione (Acceleo) non saprebbe quali di queste classi debbano diventare codice Java e quali invece rappresentino componenti di metallo e silicio (come i sensori e i motori). Applicando il Profilo UML che avrai creato nella Fase 1, tu **arricchisci** il modello sorgente. In questo modo, la tua query OCL in Acceleo potrà filtrare facilmente gli elementi, dicendo al motore: "Genera il file di testo *solo* per le classi che possiedono lo stereotipo «software»".

UML Profile: example



Input UML Model



5.4 Requisiti della Trasformazione M2T (Esempio Pratico)

Prima di iniziare a scrivere il codice MTL, è necessario definire chiaramente i requisiti della trasformazione. In questo esempio basato su un dominio accademico, le direttive sono le seguenti:

- Il **modello UML sorgente** (*source UML model*) deve essere preventivamente esteso con uno specifico profilo denominato *FacultyProfile*.
- La trasformazione M2T dovrà generare un'implementazione **Java** del modello sorgente (questa prende il nome di **target implementation** o implementazione di destinazione).
- L'implementazione di destinazione dovrà prendere in considerazione **esclusivamente** le entità relative alle attività di ricerca. Nello specifico, la trasformazione dovrà filtrare ed elaborare solo i package marcati con lo stereotipo «**Department**» e le classi marcate con lo stereotipo «**FacultyMember**».
- Le associazioni presenti nel diagramma UML dovranno essere **ignore** durante la generazione del codice.
- Nell'implementazione di destinazione, le classi che rappresentano i professori ordinari (**full professors**) dovranno includere un **nuovo attributo budget**, il quale non era fisicamente presente nel modello UML di partenza.

Confronto Analitico: Esempio Slide vs. Progetto Satellite

Analizzare i requisiti di questa slide è l'esercizio perfetto per prepararti al tuo progetto. Vediamo il parallelismo e le **differenze chiave**:

1. **Profilo UML:** La slide richiede il *FacultyProfile*. Nel tuo progetto, dovrai creare un profilo personalizzato per il controllo d'assetto con gli stereotipi «hardware» e «software».
2. **Filtro delle Entità:** L'esempio genera codice solo per dipartimenti e docenti. Il tuo script Acceleo dovrà scansionare il modello e generare i file `.java` **limitatamente ai soli componenti software** (ignorando i sensori e la Reaction Wheel).
3. **Associazioni (ATTENZIONE!):** L'esempio didattico ti dice di *ignorare* le associazioni UML. Al contrario, il tuo Task richiede esplicitamente di **considerare opportunamente le relazioni di associazione ed ereditarietà**. Questo significa che il tuo template MTL dovrà estrarre e tradurre anche i collegamenti tra le classi (es. attributi di tipo classe esterna).
4. **Aggiunte esterne al Modello:** La slide aggiunge un "budget" non previsto. Il tuo task richiede di estrarre l'attributo `note` (dal Tagged Value del profilo) e stamparlo come **commento (Javadoc)** alla classe Java generata.